

3MICT

	с.
Вступ.....	6
1 Постановка задачі.....	7
2 Основна частина.....	8
2.1 Аналіз вхідних та вихідних даних.....	8
2.2 Методи та засоби програмування	9
2.3 Опис логічної структури програми.....	18
2.4 Опис фізичної структури програми.....	28
2.5 Тестування програми.....	38
2.6 Вимоги до складу та параметрів технічних та програмних засобів.....	47
2.7 Інструкція користувача програми.....	48
2.8 Результати реалізації програми.....	53
Висновки по роботі.....	54
Перелік використаних джерел.....	55
Додаток А (Блок-схеми).....	56
Додаток Б (Лістинг програми).....	65

					ФКЗЕ.121ООП11.КРПЗ								
Змін	Арк	№ докум	Підпис	Дата	Розробити за паттерном «Декоратор» програмну систему для додавання функціональності до текстових повідомлень					Літера	Аркуш	Аркушів	
Розроб.		Кудрявець В.В.										5	78
Перевір.		Логвіненко В.В.											
Н.Контр		Логвіненко В.В.											
Затв.		Саприкіна І.Г.								група ПЗ-22-1/9			

ВСТУП

У сучасному інформаційному суспільстві комунікація є одним із ключових аспектів діяльності людей. Щодня мільярди текстових повідомлень передаються через електронну пошту, месенджери, форуми та інші цифрові платформи. Зручність спілкування, швидкість обміну інформацією та можливість структурованого представлення тексту є важливими аспектами розвитку таких технологій.

Актуальною проблемою є забезпечення гнучкості та розширюваності текстових повідомлень. У багатьох застосунках користувачам необхідно не лише вводити текст, а й формувати його - наприклад, виділяти ключові слова жирним шрифтом, додавати курсив або змінювати стиль написання. У зв'язку з цим виникає потреба у створенні програмних рішень, які дозволяють легко змінювати вигляд тексту, не порушуючи загальної логіки роботи системи.

Не менш важливим є ефективне збереження та управління текстовими повідомленнями. У великих системах, таких як корпоративні месенджери чи сервіси електронної пошти, необхідно не лише організувати дані, а й забезпечити їхню унікальність та швидкий доступ до необхідної інформації.

Метою курсової роботи є розробка програми з використанням класів - Повідомлення, BoldMessageDecorator, ItalicMessageDecorator, застосовуючи концепції ООП, а саме - успадкування класів та поліморфізм. Також для реалізації завдання будуть використані динамічні масиви даних стандартної бібліотеки шаблонів STL-контейнер set та патерн ОО проєктування – «Декоратор».

1 ПОСТАНОВКА ЗАДАЧІ

Розробити контейнерний клас для збереження та обробки масиву текстових повідомлень за патерном ООП проектування «Декоратор». Даний підхід дозволить додавати нові можливості форматування тексту без зміни його базової структури, що забезпечує гнучкість та масштабованість програмного рішення.

Для реалізації системи необхідно розробити наступні компоненти:

- 1) абстрактний клас Message - базовий клас для текстових повідомлень;
- 2) похідні від базового класу декоратори;
 - BoldMessageDecorator - декоратор для додавання жирного форматування до тексту;
 - ItalicMessageDecorator - декоратор для застосування курсиву.
- 3) контейнерний клас «MessageStorage» для збереження текстових повідомлень та їх обробки;
- 4) функціонал пошуку повідомлень за ключовими словами та іншими критеріями;
- 5) механізм збереження та завантаження повідомлень із файлу для забезпечення довготривалого зберігання даних.

Для роботи з масивом об'єктів використовується контейнерний клас set, що забезпечує унікальність текстових повідомлень.

Програма курсової роботи повинна мати наступний функціонал:

- форматування тексту - можливість застосовувати різні стилі оформлення повідомлень (жирний, курсив тощо);
- збереження та зчитування повідомлень із зовнішнього файлу;
- меню для роботи з контейнером - у програмі має бути реалізований інтерфейс для керування повідомленнями (відкриття файлу, збереження, редагування, додавання, видалення записів, пошук, друк тощо).

2 ОСНОВНА ЧАСТИНА

2.1 Аналіз вхідних та вихідних даних

Розроблена програма для роботи з текстовими повідомленнями за патерном «Декоратор» дозволяє застосовувати різні стилі форматування, здійснювати пошук повідомлень за ключовими словами та зберігати дані у файл. Для коректної роботи система приймає певні вхідні дані та генерує відповідні вихідні результати.

Вхідні дані:

- текст повідомлення - рядок, введений користувачем для збереження або редагування;
- вибір дії у меню - номер пункту меню, який визначає операцію (створення повідомлення, форматування, пошук, видалення тощо);
- застосування форматування - вибір стилю (жирний, курсив) для конкретного повідомлення;
- ключові слова для пошуку - текстові фрагменти, що використовуються для фільтрації повідомлень у контейнері;
- файл із збереженими повідомленнями - зовнішній файл, з якого завантажуються раніше збережені повідомлення.

Вихідні дані:

- відформатований текст повідомлень - виведений на екран з урахуванням застосованих стилів форматування;
- список доступних операцій - меню з можливими діями користувача (додавання, редагування, видалення повідомлень тощо);
- результати пошуку - список повідомлень, що відповідають критеріям;
- збереження повідомлень у файл - можливість експортування всіх повідомлень разом із форматуванням;
- системні повідомлення - підтвердження успішного виконання операцій або повідомлення про помилки.

2.2 Методи та засоби програмування

Для виконання та реалізації поставленого завдання курсової роботи будуть застосовані такі розділи програмування ООП мовою C++, як:

- класи, інкапсуляція;
- успадкування;
- зв'язки: узагальнення, реалізація, агрегація, залежність;
- поліморфізм;
- контейнер STL - set;
- ітератори;
- патерн проектування «Декоратор»;
- файлові потоки вводу-виводу.

2.2.1 Об'єктно-орієнтований підхід до розроблення програмних продуктів побудований на такому понятті як класи. Клас - це базова одиниця інкапсуляції (поєднання даних і дій над ними в єдине ціле), яка забезпечує механізм побудови об'єктів [2].

Терміни приховання та інкапсуляція даних є ключовими в описі об'єктно-орієнтованих мов програмування. Інкапсуляцією (encapsulation) називають способи вбудовування сутностей простішої природи - складових частин або компонентів - в агрегати, що утворюють із них нові, складніші сутності [1].

Інкапсуляція реалізується через механізм класів, які дозволяють приховати внутрішні дані та обмежити доступ до них. Для цього використовуються модифікатори доступу:

- private - члени класу доступні лише всередині самого класу;
- protected - члени класу доступні в межах класу та його нащадків;
- public - члени класу доступні ззовні.

Інкапсуляція відіграє важливу роль у зв'язках між класами. Вона дозволяє приховувати деталі реалізації одного класу від іншого, забезпечуючи взаємодію лише через визначені методи [7].

2.2.2 Успадкування - один з трьох фундаментальних механізмів об'єктно-

Змін	Арк	№ докум	Підпис	Дата

ФКЗЕ.121ООП11.КРПЗ

Арк
9

орієнтованого програмування, оскільки саме завдяки йому уможлиблюється створення ієрархічних класифікацій.

У стандартній термінології мови програмування C++ початковий клас називається базовим. Клас, який успадковує базовий клас, називається похідним [1].

Успадкування реалізації дає змогу будувати ієрархії класів; кожен дочірній клас успадковує функціональність від своїх батьків. Вкладені класи винятків утворюють приклад ієрархії класів і являють собою дизайн успадкування реалізації.

Похідні класи оголошуються з використанням такого синтаксису:

```
struct DerivedClass : BaseClass {  
};
```

Щоб визначити відносини успадкування для DerivedClass, використовується двокрапка (:), за якою слідує ім'я базового класу BaseClass. Похідні класи оголошуються так само, як і будь-який інший клас [3].

Кодова повторюваність зменшується: Спільний функціонал визначається в базовому класі, а похідні його використовують.

Гнучкість та масштабованість - легко додавати нові класи, не змінюючи код базових класів.

Поліморфізм - можливість перевизначення методів дозволяє реалізовувати різні варіанти поведінки.

Таким чином, успадкування є важливим механізмом об'єктно-орієнтованого програмування, який дозволяє будувати ієрархії класів, повторно використовувати код та реалізовувати поліморфізм у програмних системах (див. рис. 2.1) [4].

2.2.3 У об'єктно-орієнтованому програмуванні важливу роль відіграють зв'язки між класами, які визначають спосіб взаємодії об'єктів у системі. Завдяки правильно побудованим зв'язкам можна досягти високої гнучкості, повторного використання коду та зручності підтримки програмного забезпечення.

Змін	Арк	№ докум	Підпис	Дата

ФКЗЕ.121ООП11.КРПЗ

Арк
10

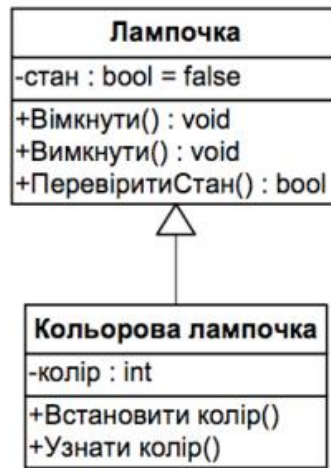


Рисунок 2.1 - Приклад спадкування

До основних типів зв'язків між класами належать узагальнення, реалізація, агрегація, композиція та залежність.

2.2.3 Узагальнення - відношення спеціалізації/узагальнення, в якому об'єкти спеціалізованого елемента (нащадка) можуть замінювати узагальнений елемент (предка) (див. рис. 2.2) [4].



Рисунок 2.2 - Відношення узагальнення

Реалізація - семантичне відношення між класифікаторами, де класифікатор визначає контракт, який другий класифікатор зобов'язується виконувати (див. рис. 2.3). Відносини реалізації застосовують у двох випадках: між інтерфейсами і класами (або компонентами), що реалізують їх; між прецедентами і кооперації, які реалізують їх [4].

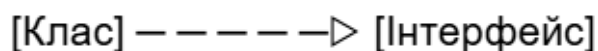


Рисунок 2.3 - Відношення реалізації

Структурна ієрархія «частина-ціле». Структурна ієрархія побудована за принципом «частина-ціле» називається агрегацією (див. рис. 2.4). Агрегація є найпростішою реалізацією однієї з базових ідей, що лежать в основі ООП: створений клас в ідеалі повинен являти собою фрагмент корисного коду, доступного для повторного використання. Проста агрегація на відміну від композитної агрегації припускає, що одна і та ж «частина» може одночасно належати різним «цілим» [7].



Рисунок 2.4 - Відношення агрегації

Залежність - семантичне відношення між двома предметами, в якому зміна в одному предметі може впливати на семантику іншого предмета (див. рис. 2.5) [4]. Іншими словами, якщо один клас (клієнт) залежить від іншого (постачальника), то зміни в інтерфейсі або поведінці постачальника можуть вимагати змін у клієнті.

Залежність є слабким типом зв'язку, що не передбачає володіння. Це дозволяє зменшити ступінь зв'язаності компонентів системи, полегшуючи тестування, повторне використання та підтримку коду.



Рисунок 2.5 - Відношення залежності

2.2.4 Однією з трьох основних особливостей об'єктно-орієнтованого програмування є поліморфізм. Стосовно мови програмування C++, під терміном поліморфізм розуміють механізм реалізації функції, у якому різні результати можна отримати за допомогою одного її імені. З цієї причини поліморфізм іноді характеризується фразою "один інтерфейс, багато методів". Це означає, що до

всіх функцій-членів класу можна отримати доступ одним і тим самим способом, незважаючи на можливу відмінність у конкретних діях, пов'язаних з кожною окремою операцією.

У мові програмування C++ поліморфізм підтримується як у процесі виконання програми, так у період її компілювання [1].

2.2.5 Стандартна бібліотека шаблонів (STL) - це частина `stdlib`, яка надає контейнери та алгоритми для керування ними, а ітератори слугують інтерфейсом між ними. У наступних трьох розділах ви дізнаєтеся більше про кожен із цих компонентів.

Контейнер - це спеціальна структура даних, яка зберігає об'єкти організованим чином, дотримуючись певних правил доступу. Існує три види контейнерів:

- у контейнери послідовності зберігають елементи послідовно, як у масиві;
- у асоціативні контейнери зберігають відсортовані елементи;
- у невідсортовані асоціативні контейнери зберігають хешовані об'єкти.

Контейнерний клас `set`, який входить до стандартної бібліотеки шаблонів C++ (STL) і підключається через заголовковий файл `<set>`, є асоціативним контейнером. Він забезпечує зберігання відсортованих унікальних елементів, що використовуються як ключі (див. рис. 2.6) [5]. Оскільки `set` зберігає відсортовані елементи, можна ефективно вставляти, видаляти та шукати елементи. Крім того, `set` підтримує відсортовану ітерацію за його елементами, і можна повністю контролювати, як ключі сортуються за допомогою об'єктів порівняння [6].

Операції над множинами виконуються швидко, тому що множини зазвичай реалізуються як червоно-чорні дерева. Ці структури обробляють кожен елемент як вузол. Кожен вузол має одного батька і до двох дітей, його ліву і праву ноги. Дочірні елементи кожного вузла сортуються, тому всі дочірні елементи ліворуч менші за дочірні елементи праворуч. Таким чином, можна виконувати пошук набагато швидше, ніж за допомогою лінійної ітерації, якщо гілки дерева приблизно збалансовані (рівні за довжиною). Червоно-чорні дерева мають додаткові можливості для відновлення балансу гілок після вставок і видалень.

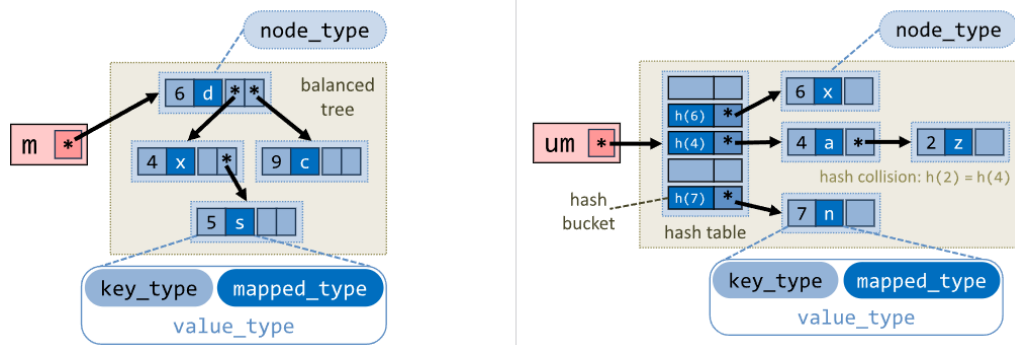


Рисунок 2.6 - Зберігання ключів пов'язаних вказівниками у контейнерному класі set

Основні операції [6]:

- begin() - Повертає ітератор, що вказує на перший елемент;
- end() - Повертає ітератор, що вказує на наступний елемент після останнього;
- find(t) Повертає ітератор, що вказує на елемент, відповідний t або end(), якщо такого елемента не існує;
- clear() - Видаляє всі елементи з набору;
- erase(t) - Видаляє елемент, що дорівнює t;
- insert(t) - Вставляє копію t у множину t;
- empty() - Повертає true, якщо розмір множини дорівнює нулю; в іншому випадку - false;
- size() - Повертає кількість елементів у множині s.extract(t);
- extract(itr) - Отримує дескриптор вузла, якому належить елемент що відповідає t або на який вказує itr. (Це єдиний спосіб видалити елемент тільки для переміщення;
- swap(s2) - Замінює всі елементи s1 на елементи s2.

2.2.6 Ітератори - це об'єкти, які тією чи іншою мірою діють подібно до покажчиків. Вони дають змогу циклічно здійснювати запит до вмісту контейнера практично так само, як це робиться за допомогою покажчика під час циклічного доступу до елементів масиву. Існує п'ять типів ітераторів (див. табл. 2.1) [2].

Таблиця 2.1 - Типи ітераторів

Ітератори	Опис
Довільного доступу (random access)	Зберігають і зчитують значення; дають змогу організувати довільний доступ до елементів контейнера
Двонаправлені (bi-directional)	Зберігають і зчитують значення; забезпечують інкрементно-декрементне переміщення
Однонаправлені (forward)	Зберігають і зчитують значення; забезпечують тільки інкрементне переміщення елементами контейнера
Вхідні (GetPut)	Зчитують, але не записують значення; забезпечують
Вихідні (output)	Записують, але не зчитують значення; забезпечують

Ітератори - обробляються аналогічно до покажчиків. Їх можна інкрементувати і декрементувати. До них можна застосовувати оператор перейменування адреси (*). Ітератори оголошуються за допомогою типу `iterator`, який визначається різними контейнерами. Бібліотека STL підтримує реверсивні ітератори, які є або двонаправленими, або ітераторами довільного доступу, даючи змогу переміщатися елементами послідовності у зворотному порядку. Отже, якщо реверсивний ітератор вказує на кінець послідовності, то після інкрементування він вказуватиме на елемент, розташований перед кінцем послідовності.

2.2.7 Декоратор - це структурний патерн проектування, що дає змогу додавати об'єктам нову функціональність, загортаючи їх у корисні «обгортки». «Декоратор» на прикладі системи обробки даних. У центрі знаходиться інтерфейс `DataSource`, який визначає базову поведінку - методи `writeData(data)` і `readData()`. Конкретна реалізація цього інтерфейсу - клас `FileDataSource`, який виконує зчитування та запис інформації у файл. Щоб розширити функціональність об'єкта без зміни його структури, використовується абстрактний клас `DataSourceDecorator`, який також реалізує інтерфейс `DataSource` і зберігає посилання на інший об'єкт цього ж типу. Від нього успадковуються конкретні декоратори - `EncryptionDecorator` і `CompressionDecorator`, які додають відповідно шифрування та стиснення даних. Клієнт взаємодіє з інтерфейсом `DataSource`, не знаючи, скільки декораторів було застосовано. Така структура дозволяє комбінувати додаткові можливості та динамічно змінювати поведінку об'єктів. (див. рис. 2.7) [6].

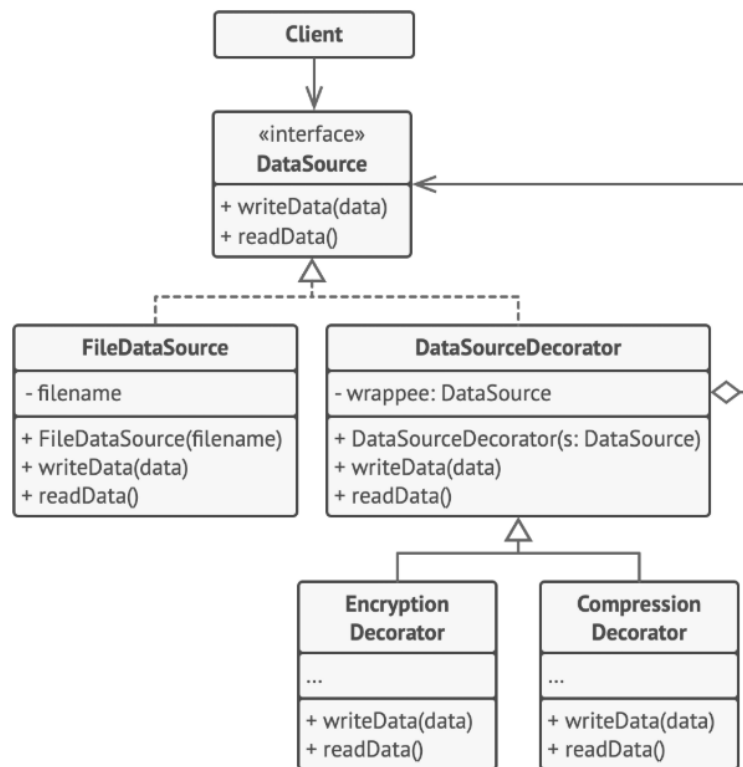


Рисунок 2.7 - Схема побудови класів патерну та зв'язки між ними

Патерн "Декоратор" дозволяє гнучко і динамічно розширювати функціональність об'єктів, не змінюючи їхній код. Він дає можливість поєднувати різні "декорації" у будь-якому порядку, уникаючи великої кількості підкласів.

Основні переваги:

- 1) додає нову поведінку без зміни існуючого коду - важливо, коли ти не можна змінити базовий клас;
- 2) зменшує кількість підкласів - замість створення десятків варіантів класів (наприклад, Повідомлення з курсивом і жирним);
- 3) дозволяє комбінувати декоратори - один і той самий об'єкт можна одночасно зробити і курсивним, і жирним;
- 4) працює з принципами SOLID;
 - відповідає принципу відкритості/закритості: класи відкриті для розширення, але закриті для змін;
 - дотримується принципу єдиної відповідальності - кожен декоратор

відповідає лише за одну зміну (жирний, курсив тощо).

2.2.8 У C++ - системі введення-виведення також передбачено засоби для виконання відповідних операцій з використанням файлів. Файлові операції введення-виведення даних можна реалізувати після внесення у програму заголовка `<fstream>`, у якому визначено всі необхідні для цього класи і значення.

У мові програмування C++ файл відкривається шляхом зв'язування його з потоком. Як уже зазначалося вище, існують потоки трьох типів: введення, виведення і введення-виведення. Щоб відкрити вхідний потік, необхідно оголосити потік класу `ifstream`. Для відкриття вихідного потоку потрібно оголосити потік класу `ofstream`. Потік, який передбачається використовувати для операцій як введення, так і виведення, повинен бути оголошений як потік класу `fstream`. Наприклад, у процесі виконання такого фрагмента коду програми буде створено вхідний і вихідний потоки, а також потік, що дає змогу виконувати операції в обох напрямках.

Створивши потік, його потрібно пов'язати з файлом. Це можна зробити за допомогою функції `open()`, причому у кожному з трьох потокових класів є своя функція-член `open()`. Їх прототипи мають такий вигляд:

```
void ifstream::open(const char *filename, ios::openmode mode = ios::in);  
void ofstream::open(const char * filename, ios::openmode mode = ios::out |  
ios::trunc);  
void fstream::open(const char *filename, ios::openmode mode = ios::in | ios::out);
```

У цих записах елемент `filename` означає ім'я файлу, яке може містити специфікатор шляху, що вказує доступ до нього. Елемент `mode` називається специфікатором режиму, який визначає спосіб відкриття файлу. Він повинен приймати одне або декілька значень перерахунку `openmode`, який визначено у класі `ios` [2]:

- `ios::app` - приєднує до кінця файлу усі дані, що виводяться;
- `ios::binary` - відкриває файл у двійковому режимі;
- `ios::in` - забезпечує відкриття файлу для введення даних;
- `ios::out` - забезпечує відкриття файлу для виведення даних;

Змін	Арк	№ докум	Підпис	Дата

ФКЗЕ.121ООП11.КРПЗ

Арк
17

2.3 Опис логічної структури програми

Для виконання завдання курсової роботи була спроектована та реалізована ієрархія класів: Message, SimpleMessage, MessageDecorator, BoldMessageDecorator, ItalicMessageDecorator, а також контейнерний клас MessageStorage. Діаграма класів та зв'язки між ними наведені на рисунку (див. рис. 2.8).

Класи створюються для зберігання та обробки повідомлень у чаті. Розроблені класи мають таке призначення:

Message — абстрактний базовий клас, що визначає спільний інтерфейс для всіх типів повідомлень. Він зберігає текст повідомлення та унікальний ідентифікатор ID, а також надає віртуальні методи для доступу до даних і виведення. Клас забезпечує можливість впорядкування повідомлень у контейнерах за допомогою перевантаженого оператора <.

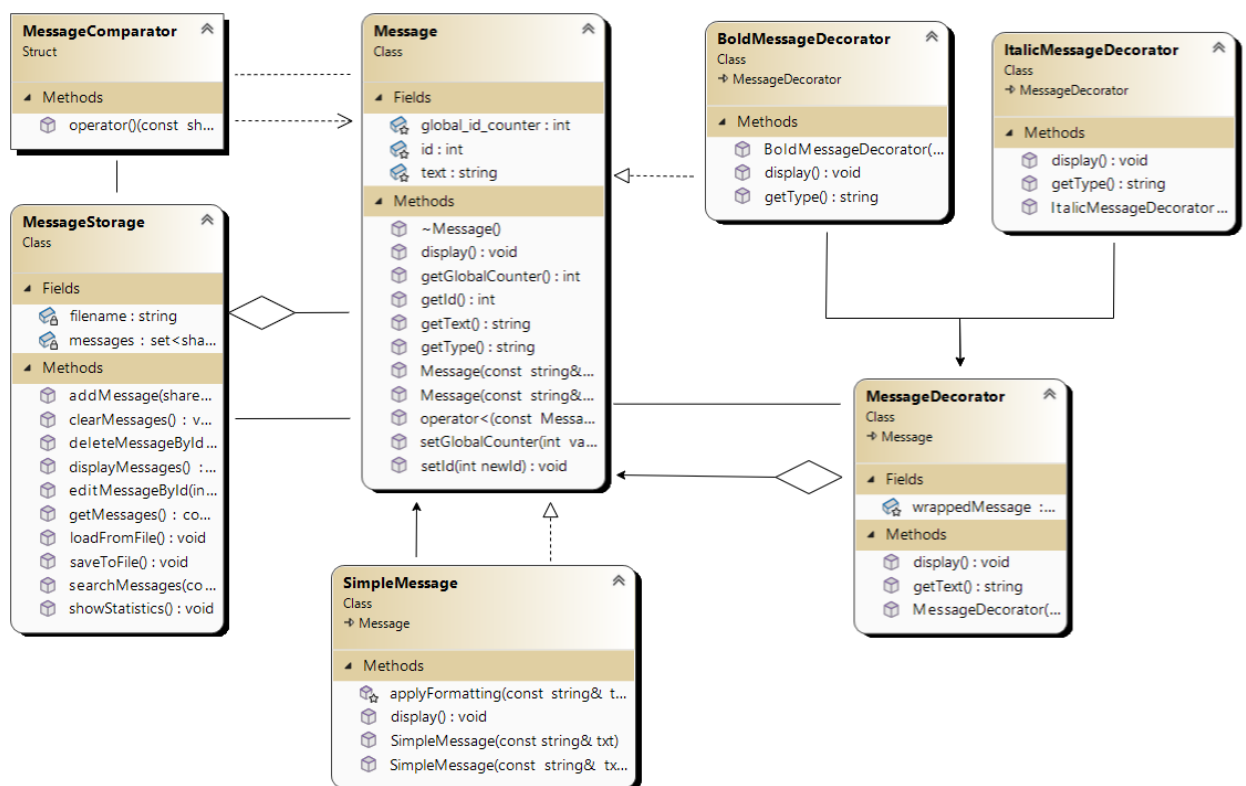


Рисунок 2.8 - Діаграма класів Message, SimpleMessage, MessageDecorator, BoldMessageDecorator, ItalicMessageDecorator

- SimpleMessage - клас, що наслідує Message та реалізує базовий функціонал відображення повідомлень. У тексті підтримується умовне форматування жирного (текст) та курсиву (текст);

- MessageDecorator - абстрактний клас-декоратор, що наслідує Message та дозволяє змінювати вигляд виведення без зміни внутрішньої структури повідомлення;

- BoldMessageDecorator - конкретна реалізація декоратора, що змінює вигляд повідомлення, стилізуючи його як жирний текст;

- ItalicMessageDecorator - конкретна реалізація декоратора, що стилізує повідомлення курсивом;

- MessageStorage - контейнерний клас, у якому використовується STL-контейнер `set<shared_ptr<Message>>` з компаратором `MessageComparator` для зберігання унікальних повідомлень. У класі реалізовано повний набір функцій: додавання, редагування, видалення повідомлень, пошук, збереження та завантаження з файлу, очищення чату, перегляд статистики.

Усі повідомлення зберігаються у вигляді об'єктів `shared_ptr<Message>` із забезпеченням унікальності ID. Програма підтримує взаємодію через консольне меню, що дозволяє користувачу виконувати необхідні дії над повідомленнями.

Структура визначення класів складається з властивостей/атрибутів та методів, які можуть мати різний рівень доступу (захисту), такі як: закриті (private), захищені (protected), відкриті (public).

Так, у якості атрибутів класу Message виступають базові типи даних:

- `static int global_id_counter` - статична змінна, що зберігає останній присвоєний ідентифікатор повідомлення;

- `int id` - ціла змінна збереження унікального ідентифікатора повідомлення;

- `string text` - символьний рядок збереження тексту повідомлення.

Оголошені методи класу Message мають відповідну сигнатуру та наступне призначення:

- `Message(const string& txt)` - конструктор класу, що приймає текст повідом-

лення і автоматично присвоює ID;

- `Message(const string& txt, int forcedId)` - конструктор класу з параметрами тексту повідомлення та заданим ID (наприклад, при завантаженні з файлу);

- `virtual ~Message()` - віртуальний деструктор класу, необхідний для коректного наслідування;

- `void setId(int newId)` - встановлює нове значення ID для повідомлення;

- `int getId() const` - повертає унікальний ідентифікатор повідомлення;

- `static int getGlobalCounter()` - повертає поточне значення глобального лічильника ID;

- `static void setGlobalCounter(int value)` - встановлює нове значення глобального лічильника ID;

- `virtual string getText() const` - повертає текст повідомлення;

- `virtual string getType() const` - повертає тип повідомлення (за замовчуванням - «Просте»);

- `virtual void display() const` - виводить повідомлення у консоль у форматі: ID та текст.

Функції індивідуального доступу до даних членів класу `Message`:

- встановлення (set) значень атрибутам класу: `void setId(int newId); static void setGlobalCounter(int value);`

- отримання (get) значень атрибутів класу: `int getId(); static int getGlobalCounter(); string getText(); string getType();`

Перевантажені базові оператори для класу `Message` - оператор порівняння (<) використовується в контейнерах (наприклад, `set`) для порівняння повідомлень за ID.

Клас `SimpleMessage` є конкретною реалізацією повідомлення. Він успадковує функціональність базового класу `Message` та реалізує спеціалізований вивід повідомлень у консоль з підтримкою умовного форматування жирного (текст) та курсиву (текст).

Так, у якості атрибутів класу `SimpleMessage` використовується успадкована

Змін	Арк	№ докум	Підпис	Дата

ФКЗЕ.121ООП11.КРПЗ

Арк

20

структура з базового класу Message:

- int id - унікальний ідентифікатор повідомлення;
- string text - символьний рядок збереження тексту повідомлення.

Оголошені методи класу SimpleMessage мають відповідну сигнатуру та наступне призначення:

- SimpleMessage(const string& txt) - конструктор класу з автоматичним при-
своєнням ID;
- SimpleMessage(const string& txt, int forcedId) - конструктор класу з явним ID
(використовується при завантаженні з файлу);
- void display() const override - перевизначений метод виведення повідомлення
у консоль. Реалізує вивід з обробкою символів форматування (* - жирний, _ - кур-
сив).

Функції індивідуального доступу до даних членів класу SimpleMessage не оголошуються окремо, оскільки використовуються методи доступу, успадковані з базового класу Message.

Власні (додаткові) методи класу SimpleMessage:

void applyFormatting(const string& text) const - допоміжна захищена функція, яка відповідає за інтерпретацію символів форматування (*, _) у тексті повідомлення та зміну кольору/стилю виводу відповідно.

Перевантажені базові оператори не оголошуються безпосередньо в класі SimpleMessage, оскільки успадковуються з базового класу Message, зокрема:

bool operator<(const Message& other) const; - оператор порівняння повідомлень за ID.

Клас MessageDecorator є абстрактним класом-декоратором, що успадковує клас Message і дозволяє змінювати зовнішній вигляд виведення повідомлення, не змінюючи його внутрішньої структури. Реалізує шаблон проєктування «Декоратор».

Так, у якості атрибутів класу MessageDecorator виступають такі змінні:

shared_ptr<Message> wrappedMessage - покажчик на об'єкт повідомлення,

який обгортається (декорується).

Оголошені методи класу MessageDecorator мають відповідну сигнатуру та наступне призначення:

- MessageDecorator(shared_ptr<Message> msg) - конструктор класу, що приймає як параметр вказівник на об'єкт повідомлення (Message) та ініціалізує значення id і text через базовий клас Message;

- virtual string getText() const override - повертає текст повідомлення з обгорнутого (wrappedMessage) об'єкта;

- virtual void display() const override - віртуальний метод виведення повідомлення, який делегує відображення вкладеному об'єкту wrappedMessage.

Функції індивідуального доступу до даних членів класу MessageDecorator не оголошуються окремо, оскільки використовуються геттер/сеттер з базового класу Message.

Перевантажені базові оператори для класу MessageDecorator не оголошуються окремо, але успадковуються з класу Message:

- bool operator<(const Message& other) const - оператор порівняння за ID, що дозволяє використовувати об'єкти класу MessageDecorator у впорядкованих контейнерах (set тощо).

Клас BoldMessageDecorator є конкретною реалізацією декоратора, що наслідує MessageDecorator. Він змінює спосіб виведення повідомлення у консоль, стилізуючи його як жирний текст (через зміну кольору фону тексту).

Так, у якості атрибутів класу BoldMessageDecorator не оголошуються додаткові змінні, оскільки всі необхідні поля наслідуються з класів Message та MessageDecorator:

- shared_ptr<Message> wrappedMessage - об'єкт, що декорується;

- int id, string text - атрибути, ініціалізовані з базового повідомлення.

Оголошені методи класу BoldMessageDecorator мають відповідну сигнатуру та наступне призначення:

- BoldMessageDecorator(shared_ptr<Message> msg) - конструктор класу, який

приймає об'єкт типу Message, що буде декоровано;

- string getType() const override - метод, що повертає тип повідомлення. У цьому випадку - "Bold";

- void display() const override - перевизначений метод виведення повідомлення у консоль із форматуванням жирного тексту (через зміну кольору фону за допомогою SetConsoleTextAttribute()).

Функції індивідуального доступу до даних членів класу BoldMessageDecorator не оголошуються, оскільки використовуються методи базового класу Message.

Перевантажені базові оператори для класу BoldMessageDecorator не оголошуються явно, а успадковуються з базового класу Message, зокрема:

- bool operator<(const Message& other) const - оператор порівняння повідомлень за ID.

Клас ItalicMessageDecorator є конкретною реалізацією декоратора, що наслідує MessageDecorator. Він змінює вигляд виведення повідомлення у консоль, стилізуючи його як курсив (з використанням ANSI escape-послідовностей \033[3m).

Так, у якості атрибутів класу ItalicMessageDecorator додаткові змінні не використовуються, оскільки всі необхідні поля наслідуються з класів Message та MessageDecorator:

- shared_ptr<Message> wrappedMessage - об'єкт, що декорується;
- int id, string text - значення, отримані з базового повідомлення.

Оголошені методи класу ItalicMessageDecorator мають відповідну сигнатуру та наступне призначення:

- ItalicMessageDecorator(shared_ptr<Message> msg) - конструктор класу, що приймає об'єкт повідомлення (Message) і зберігає його для подальшої обробки;

- string getType() const override - повертає тип повідомлення, який у цьому випадку є "Italic";

- void display() const override - перевизначений метод, який реалізує виведення повідомлення у консоль з використанням курсивного стилю (\033[3m ...

Змін	Арк	№ докум	Підпис	Дата

ФКЗЕ.121ООП11.КРПЗ

Арк
23

\033[0m) та кольору тексту за замовчуванням.

Функції індивідуального доступу до даних членів класу `ItalicMessageDecorator` не оголошуються, оскільки використовуються геттер/сеттер з базового класу `Message`.

Перевантажені базові оператори для класу `ItalicMessageDecorator` не оголошуються явно, але успадковуються з базового класу `Message`:

`bool operator<(const Message& other) const` - оператор порівняння за ID для впорядкування у контейнерах типу `set`.

`MessageStorage` — контейнерний клас, що забезпечує зберігання та керування повідомленнями у вигляді `shared_ptr<Message>`. Він підтримує повний набір операцій: додавання, редагування, видалення, пошук, вивід статистики, а також збереження й завантаження з файлу. Повідомлення зберігаються у впорядкованому контейнері `set` з компаратором `MessageComparator`, що гарантує унікальність ID.

Атрибути класу `MessageStorage`:

- `set<shared_ptr<Message>, MessageComparator> messages` - впорядкований контейнер повідомлень з унікальними ID (використовується компаратор `MessageComparator`);

- `string filename` - символьний рядок, що містить назву файлу для збереження та завантаження повідомлень (за замовчуванням - "messages.txt").

Оголошені методи класу `MessageStorage` мають відповідну сигнатуру та наступне призначення:

- `void addMessage(shared_ptr<Message> msg)` - додає нове повідомлення до контейнера. Якщо повідомлення з таким ID вже існує - не додається;

- `void displayMessages() const` - виводить усі збережені повідомлення в консоль у впорядкованому вигляді;

- `void editMessageById(int id)` - дозволяє редагувати текст повідомлення за його ID. Створює новий об'єкт на місці старого з тим самим ID;

- `void deleteMessageById(int id)` - видаляє повідомлення за його ID;

- `void searchMessages(const string& keyword) const` - здійснює пошук повідом-

лень, які містять задане ключове слово, та виводить результати в консоль;

- void clearMessages() - очищає весь контейнер повідомлень;

- void showStatistics() const - виводить статистику чату: загальна кількість повідомлень, слів, символів, а також кількість фрагментів жирного (*) та курсивного () форматування;

- void saveToFile() - зберігає повідомлення у файл у вигляді: ID: <id>|<text>;

- void loadFromFile() - завантажує повідомлення з текстового файлу. Автоматично оновлює глобальний лічильник ID.

Функції індивідуального доступу:

Клас не надає окремих геттерів/сеттерів, оскільки вся взаємодія з повідомленнями здійснюється через методи роботи з контейнером.

Використання перевантажених операторів у класі MessageStorage:

У класі MessageStorage оператори не перевантажуються напряду, проте використовуються успадковані оператори operator< для об'єктів Message під час зберігання у set.

Особливості реалізації класу MessageStorage:

- забезпечується унікальність повідомлень у контейнері за допомогою set і перевантаженого оператора <;

- для відновлення стану після перезапуску передбачене збереження та завантаження даних з файлу;

- для розширення функціональності клас використовує shared_ptr, що дозволяє обробляти як прості, так і стилізовані повідомлення (Bold, Italic) через єдиний інтерфейс;

- обчислення статистики повідомлень включає обробку тексту на рівні слів, символів та форматування.

Функтор-компаратор MessageComparator:

Структура визначення MessageComparator складається з одного перевантаженого оператора та призначена для використання як функтор-компаратор у впорядкованому контейнері set. Забезпечує сортування об'єктів типу

shared_ptr<Message> за зростанням їх унікального ідентифікатора id.

Атрибутивна структура MessageComparator:

Структура MessageComparator не містить внутрішніх атрибутів, оскільки реалізує лише функціональний інтерфейс (оператор ()).

Для тестування можливостей розробленої ієрархії класів, складена програма яка має виклики методів обробки контейнера екземплярів класу MessageApp та меню виконання для їх виклику. Схема меню програми наведена на рисунку (див.рис. 2.9)

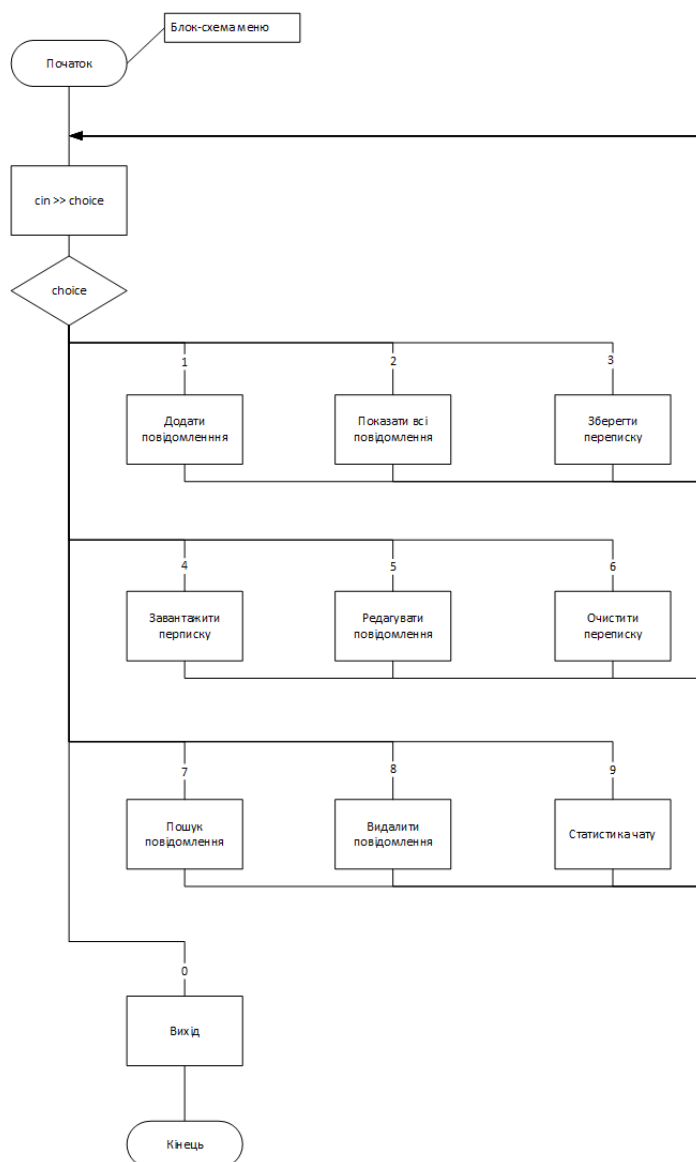


Рисунок 2.9 - Функціональна схема меню програми

Метод порівняння у структурі MessageComparator:

```
bool operator()(const shared_ptr<Message>& lhs, const shared_ptr<Message>& rhs) const
```

 - перевантаження функціонального оператора ().

Повертає true, якщо id лівого операнда менше id правого, що дозволяє контейнеру set зберігати об'єкти у впорядкованому вигляді без дублікатів.

Структура MessageComparator використовується в оголошенні контейнера повідомлень наступним чином:

```
- set<shared_ptr<Message>;  
- MessageComparator> messages;
```

Дозволяє зберігати вказівники на об'єкти різних нащадків Message у впорядкованому контейнері (set) з урахуванням значення id, а не вказівника у пам'яті.

Завдяки shared_ptr забезпечується поліморфізм і керування пам'яттю, а MessageComparator забезпечує детермінований порядок елементів.

2.4 Опис фізичної структури програми

Відповідно до постановки задачі, структури класів та логіки розробленого головного меню програми було реалізовано визначення методів класів: Message, SimpleMessage, MessageDecorator, BoldMessageDecorator, ItalicMessageDecorator, а також класу керування повідомленнями MessageStorage. Тестування можливостей зазначених класів реалізовано у межах основної програми, яка керується через консольне меню користувача.

Код визначення класів та їх методів зосереджено у файлі main.cpp, де реалізовано повну логіку додатку - включно з обробкою введення, виводу, логікою форматування повідомлень, збереженням у файл і завантаженням з нього. Скомпільований виконуваний файл створюється під назвою ChatFormatter.exe.

Код програми написаний мовою програмування C++ та містить визначення абстрактних типів даних (ADT), спадкування, шаблони декораторів, динамічну пам'ять (через shared_ptr), а також консольну взаємодію. Для повноцінного функціонування класів і логіки програми використано допоміжні бібліотеки, що

підключаються директивами препроцесора `#include`. Вони забезпечують доступ до стандартних типів, функцій і утиліт, зокрема:

- `<iostream>` - функції введення та виводу;
- `<iomanip>` - форматований вивід (наприклад, для статистики чату);
- `<set>` - контейнер `set` для зберігання повідомлень без дублювання ID;
- `<fstream>` - файловий ввід/вивід для збереження та завантаження переписки;
- `<sstream>` - робота з текстовими потоками та розбиттям повідомлень на слова;
- `<memory>` - розумні вказівники `shared_ptr` для безпечного керування об'єктами;
- `<windows.h>` - доступ до функцій операційної системи Windows для роботи з кольором тексту в консолі;
- `<vector>` - динамічний масив для обробки тимчасових колекцій повідомлень.

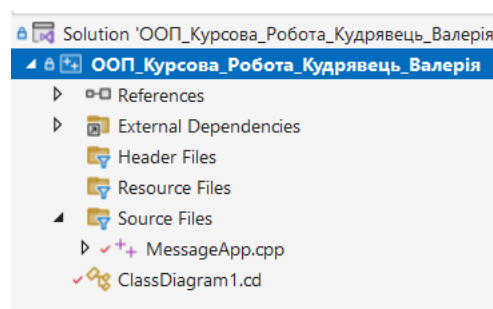


Рисунок 2.10 - Дерево файлів проєкту

Відповідно до списку атрибутів та методів кожного класу, описаних у підрозділі 4.1, було побудовано повну реалізацію програмної структури, яка базується на шаблоні проєктування «Декоратор» для гнучкого форматування тексту (жирний, курсив), та використанні контейнерів STL для збереження даних.

2.4.1 Опис методів абстрактного класу Message

Визначення методів класу father розташовані у файлі MessageApp.cpp який містить визначення конструкторів які виконують початкову ініціалізацію даних-членів класу.

Таблиця 4.1 - Параметри та змінні конструкторів класу Message

Заголовок, опис результату методу	Роль змінної	Тип	Позначення	Опис
Message(const string& txt)	Параметр	string	txt	Текст повідомлення
Message(const string& txt, int forcedId)	Параметр	string	txt	Текст повідомлення
	Параметр	int	forcedId	Призначений ID повідомлення

У класі Message визначено перевантажений оператор

Глобально (не friend) перевантажено оператор порівняння operator<, який дозволяє порівнювати об'єкти Message за значенням їхнього ID. Це забезпечує можливість впорядкування повідомлень у впорядкованих контейнерах, таких як set.

Опис перевантаженого оператора:

- bool operator<(const Message& other) const;

Порівнює два повідомлення за ID. Повертає true, якщо ID поточного об'єкта менший, ніж ID іншого.

Таблиця 2.2 - Параметри та змінні перевантажених операторних методів класу Message

Заголовок, опис результату методу	Роль змінної	Тип	Позначення	Опис
bool operator< (const Message& other) const - порівнює повідомлення за ID	Параметр	Message	other	Повідомлення для порівняння

У класі Message перевантажено оператор порівняння <, що дозволяє порівнювати об'єкти за унікальним ID. Це необхідно для використання повідомлень у

впорядкованих контейнерах, таких як set. Оператор реалізовано як метод класу, що повертає true, якщо ID поточного об'єкта менший за ID іншого.

Таблиця 4.3 - Параметри та змінні методів setX класу Message

Заголовок, опис результату методу	Роль змінної	Тип	Позначення	Опис
void setId(int newId)	Параметр	int	newId	Встановлює ID повідомлення

Методи встановлення значень властивостей (set) класу Message змінюють значення відповідного атрибута класу. Зокрема, метод setId(int newId) використовується для примусового встановлення ID повідомлення, наприклад, при завантаженні даних із файлу. У параметр методу передається нове значення, яке призначається полю id.

Методи отримання значень властивостей (get) реалізовано з використанням оператора return, який повертає значення відповідного атрибута (id, text, type). Опис методів setX наведено у таблиці 4.3, а методів getX - у таблиці 4.4.

Таблиця 4.4 - Параметри та змінні методів getX класу Message

Заголовок, опис результату методу	Роль змінної	Тип	Позначення	Опис
int getId() - повертає унікальний ідентифікатор	Параметр	void	-	Повертає ID повідомлення
string getText() - повертає текст повідомлення	Параметр	void	-	Повертає вміст повідомлення
string getType() - повертає тип повідомлення	Параметр	void	-	Повертає тип (Просте, Bold тощо)

Методами отримання значень властивостей (get) класу Message є функції, які за допомогою оператора return повертають значення відповідних полів класу. Ці методи використовуються для виводу повідомлень, побудови статистики, пошуку та відображення інформації в консольному інтерфейсі програмного застосунку.

Методи set та get разом забезпечують повну інкапсуляцію атрибутів класу та

підтримують контроль доступу до даних повідомлення.

2.4.2 Опис методів класу SimpleMessage

Визначення методів класу SimpleMessage розташовані у файлі ChatFormatter.cpp. Клас є похідним від базового класу Message і використовується для створення простих повідомлень, які можуть містити форматування у вигляді фрагментів жирного тексту (позначеного символами *****) та курсивного (позначеного символами *_*).

Клас містить два конструктори: перший - для створення нового повідомлення з автоматично присвоєним унікальним ID, другий - для створення повідомлення з явно заданим ID (наприклад, під час завантаження з текстового файлу).

Також перевизначено метод display(), який виводить повідомлення з обробкою форматування через допоміжну функцію applyFormatting(). Ця функція змінює вигляд тексту в консолі залежно від символів ***** і *_* та використовує кольорове оформлення через Windows API.

Таблиця 4.5 - Параметри та змінні конструкторів класу SimpleMessage

Заголовок, опис результату методу	Роль змінної	Тип	Позначення	Опис
SimpleMessage(const string& txt)	Параметр	string	txt	Текст повідомлення
SimpleMessage(const string& txt, int forcedId)	Параметр	string	txt	Текст повідомлення
	Параметр	int	forcedId	Призначений ID повідомлення

2.4.3 Опис методів класу MessageDecorator

Клас MessageDecorator реалізує шаблон проектування «Декоратор» і призначений для розширення базової функціональності об'єктів класу Message без зміни їхньої структури. Він містить покажчик на об'єкт Message, який обгортається і використовується у викликах методів getText() та display().

Клас має один конструктор, який приймає об'єкт повідомлення, і делегує основні виклики методів обгорнутому об'єкту. Сам MessageDecorator не змінює по-

ведінки повідомлення, але виступає базою для похідних декораторів (BoldMessageDecorator, ItalicMessageDecorator).

Таблиця 4.6 - Параметри конструктора класу MessageDecorator

Заголовок, опис результату методу	Роль змінної	Тип	Позначення	Опис
MessageDecorator(shared_ptr<Message> msg)	Параметр	shared_ptr<Message>	msg	Об'єкт повідомлення, що обгортається

2.4.4 Опис методів класу BoldMessageDecorator

Клас BoldMessageDecorator є похідним від MessageDecorator і призначений для візуального виділення повідомлень шляхом зміни кольору тексту на консолі. Він перевизначає метод display(), який виводить повідомлення з білим текстом на чорному фоні (через Windows API), тим самим імітуючи ефект "жирного" повідомлення.

Клас також перевизначає метод getType(), який повертає назву типу повідомлення - "Bold".

Таблиця 4.6 - Параметри конструктора класу BoldMessageDecorator

Заголовок, опис результату методу	Роль змінної	Тип	Позначення	Опис
BoldMessageDecorator(shared_ptr<Message> msg)	Параметр	shared_ptr<Message>	msg	Повідомлення, яке декорується жирним

2.4.4 Опис методів класу ItalicMessageDecorator

Клас ItalicMessageDecorator є нащадком класу MessageDecorator і реалізує форматування повідомлень шляхом імітації курсиву. Це реалізовано шляхом введення ANSI escape-кодів, які активують і деактивують курсивний стиль у терміналі.

Клас перевизначає метод display() для додавання ефекту курсиву, а також метод getType(), який повертає значення "Italic". Усі інші методи наслідуються від батьківських класів без змін.

Таблиця 4.7 - Параметри конструктора класу ItalicMessageDecorator

Заголовок, опис результату методу	Роль змінної	Тип	Позначення	Опис
ItalicMessageDecorator(shared_ptr<Message> msg)	Параметр	shared_ptr<Message>	msg	Повідомлення, яке декодується курсивом

2.4.5 Опис методів класу MessageStorage

Клас MessageStorage реалізує зберігання, обробку та управління колекцією повідомлень. Для збереження даних використовується контейнер типу set з об'єктами shared_ptr<Message> і компаратором MessageComparator. Клас забезпечує додавання, редагування, видалення повідомлень, пошук за ключовим словом, а також функції збереження/завантаження з текстового файлу. Додатково реалізовано метод виводу статистики по всьому чату.

Таблиця 4.8 - Опис публічних методів класу MessageStorage

Заголовок, опис результату методу	Роль змінної	Тип	Позначення	Опис
void addMessage(shared_ptr<Message> msg)	Параметр	shared_ptr<Message>	msg	Додає повідомлення до контейнера з перевіркою ID
void displayMessages()	-	-	-	Виводить усі збережені повідомлення
void editMessageById(int idToEdit)	параметр	int	idToEdit	Змінює текст повідомлення за заданим ID
void deleteMessageById(int idToDelete)	параметр	int	idToDelete	Видаляє повідомлення з колекції за ID
void saveToFile()	-	-	-	Зберігає повідомлення у текстовий файл
void loadFromFile()	-	-	-	Завантажує повідомлення з файлу у контейнер
void searchMessages(const string& keyword)	параметр	string	keyword	Шукає повідомлення, які містять задане слово
void showStatistics()	-	-	-	Виводить статистику чату (слова, символи, форматування)
void clearMessages()	-	-	-	Повністю очищує переписку з пам'яті

Публічні методи класу MessageStorage забезпечують повний цикл обробки повідомлень у чаті. Вони реалізують функції додавання, редагування, видалення,

пошуку, виводу, збереження та завантаження повідомлень, а також побудову статистики за вмістом. Метод `addMessage()` перевіряє унікальність ID перед додаванням, а `editMessageById()` дозволяє змінити текст уже наявного повідомлення.

Методи `saveToFile()` та `loadFromFile()` забезпечують роботу з локальним текстовим файлом, де зберігається історія чату. Метод `showStatistics()` підраховує загальну кількість повідомлень, слів, символів, а також кількість фрагментів з жирним і курсивним форматуванням. Всі повідомлення зберігаються у контейнері `set`, що гарантує впорядковане та унікальне зберігання.

2.4.6 Опис методу класу `MessageComparator`

Клас `MessageComparator` є функціональним об'єктом (функтором), який використовується для порівняння об'єктів типу `shared_ptr<Message>` за їхнім ID. Цей клас застосовується як параметр шаблону сортування у контейнері `set`, що забезпечує зберігання повідомлень у впорядкованому вигляді та уникнення дублікатів.

Метод класу - перевантажений оператор круглих дужок `operator()`, який порівнює ID двох об'єктів.

Таблиця 4.9 - Параметри методу `operator()` класу `MessageComparator`

Заголовок опису результату методу	Роль змінної	Тип	Позначення	Опис
<code>Bool operation()(shared_ptr<Message>lhs, shared_ptr<Message>rhs) const</code>	Параметр	<code>shared_ptr<Message></code>	lhs	Перше повідомлення для порівняння за ID
<code>Bool operator()(shared_ptr<Message>lhs, shared_ptr<Message>rhs) const</code>	Параметр	<code>shared_ptr<Message></code>	rhs	Друге повідомлення для порівняння за ID

Оператор `operator()` перевантажено для реалізації логіки порівняння двох об'єктів типу `shared_ptr<Message>` за їхнім унікальним ID. Він використовується як функціональний об'єкт-компаратор у контейнері `set`, що дозволяє впорядковувати повідомлення та забезпечувати їхню унікальність за ідентифікатором. Завдяки цьому забезпечується коректна логіка зберігання та сортування у структурі

даних класу MessageStorage.

У головній функції main() програми оголошено об'єкт класу MessageStorage - storage, який забезпечує збереження, обробку та виведення повідомлень користувача. Також виконано налаштування кодування консолі засобами Windows API для підтримки кирилиці.

Меню програми реалізовано за допомогою циклу while(true) та конструкції switch, що дозволяє обробляти вибір користувача по пунктах. Для кожного значення змінної choice, отриманого з клавіатури, виконується відповідна дія.

Кожна команда меню відповідає одному з методів класу MessageStorage:

- пункт 1 - додавання повідомлення до контейнера;
- пункт 2 - виведення всіх повідомлень;
- пункт 3 - збереження повідомлень у файл;
- пункт 4 - завантаження повідомлень із файлу;
- пункт 5 - редагування повідомлення за ID;
- пункт 6 - очищення всіх повідомлень;
- пункт 7 - пошук повідомлень за ключовим словом;
- пункт 8 - видалення повідомлення за ID;
- пункт 9 - перегляд статистики чату;
- пункт 0 - вихід з програми з можливістю збереження даних.

Вивід самого меню реалізовано окремою функцією showMenu(), яка оформлює графічне представлення пунктів дій для користувача. Після кожної дії меню оновлюється, щоб забезпечити циклічну роботу та зручну взаємодію.

- cout << "1 Додати повідомлення " << endl;
- cout << "2 Показати всі повідомлення " << endl;
- cout << "3 Зберегти переписку " << endl;
- cout << "4 Завантажити переписку " << endl;
- cout << "5 Редагувати повідомлення " << endl;
- cout << "6 Очистити переписку " << endl;
- cout << "7 Пошук повідомлення " << endl;

```
- cout << "8 Видалити повідомлення " << endl;
- cout << "9 Статистика чату " << endl;
- cout << "0 Вихід " << endl.
```

Вибір пункту меню здійснюється за допомогою введення відповідного номера, який зчитується оператором `cin >> choice;`. Далі відбувається обробка вибору користувача за допомогою оператора `switch-case`. Нижче наведено опис дій, які виконує програма залежно від обраного пункту:

- “1” - викликається функція `addMessageFlow(storage);`
- “2” - спочатку викликається функція `refreshMenu()`, а потім метод `storage.displayMessages();`
- “3” - спочатку викликається функція `refreshMenu()`, а потім метод `storage.saveToFile();`
- “4” - спочатку викликається функція `refreshMenu()`, а потім метод `storage.loadFromFile();`
- “5” - викликається функція `editMessageFlow(storage);`
- “6” - спочатку викликається функція `refreshMenu()`, а потім метод `storage.clearMessages();`
- “7” - викликається функція `searchMessageFlow(storage);`
- “8” - викликається функція `deleteMessageFlow(storage);`
- “9” - спочатку викликається функція `refreshMenu()`, а потім метод `storage.showStatistics();`
- “0” - викликається функція `exitFlow(storage)`, яка повертає `true`, якщо користувач підтверджує вихід з програми (`return 0`).

Значення, яке не відповідає ніякому з `case`: `default`: `cout << "Некоректний вибір, спробуйте знову!" << endl;`

```
} while (choice != 0);
```

Лістинг програми наведений у Додатку Б.

2.5 Тестування програми

Під час тестування програми було перевірено роботу всіх пунктів меню створеної програми, а також протестовано реалізовану ієрархію класів, об'єднаних у контейнер типу set. Окрім того, були протестовані різні варіанти значень, які використовувалися для ініціалізації атрибутів розроблених класів.

2.5.1 Тестування головного меню програми

У меню представляється 10 пунктів до вибору, номер кожного позначен відповідною цифрою зліва. При некоректному вводі виводиться відповідне повідомлення (див. рис. 2.11).

```
Виберіть дію: -1
Введіть число в межах від 0 до 9
Виберіть дію: 1212
Введіть число в межах від 0 до 9
Виберіть дію: ййй
Некоректний вибір. Введіть число від 0 до 9
Виберіть дію: qqq
Некоректний вибір. Введіть число від 0 до 9
Виберіть дію: &*^%&
Некоректний вибір. Введіть число від 0 до 9
Виберіть дію:
Некоректний вибір. Введіть число від 0 до 9
```

Рисунок 2.11 - Спроба некоректно ввести пункт головного меню

2.5.2 Перший пункт меню “Додати повідомлення”. При вводі “1” програма повідомляє про можливість при додаванні повідомлення зробити форматування тексту та ввести текст. Для того щоб завершити повідомлення введіть “/0” на наступному рядку (див. рис. 2.12).

При спробі відправити порожнє повідомлення програма виконує перевірку введеного рядка. Якщо повідомлення порожнє, користувач бачить відповідне повідомлення про помилку (див. рис. 2.13).

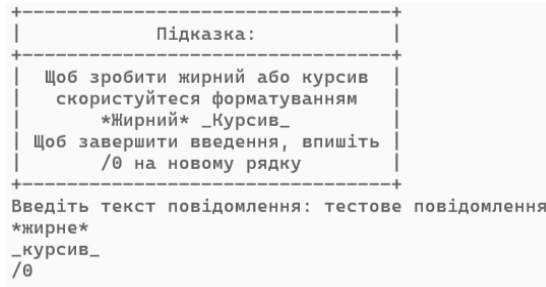


Рисунок 2.12 - Пункт меню “1” - “Додати повідомлення”

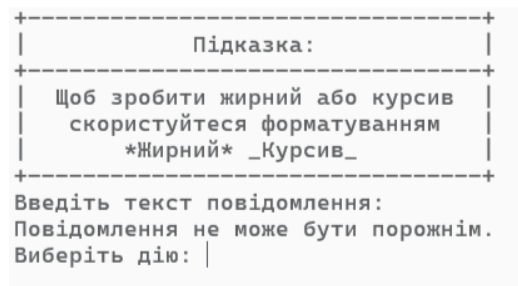


Рисунок 2.13 - Спроба ввести порожнє повідомлення

2.5.2 Другий пункт меню “Показати всі повідомлення”. При вводі “2” на екрані з'являються всі повідомлення які було раніше додано або завантажено (див. рис 2.14).

ID: 1 – тестове повідомлення
жирне
курсив

ID: 2 – Звіт про помилку:
1. Не відкривається файл
2. Виводиться повідомлення про помилку
Рішення: перевірити права доступу

ID: 3 –
Термінове повідомлення!
Сервер впав о 14:03
Причина встановлюється

ID: 4 –
Привіт усім!
Зустріч переноситься на п'ятницю.
Деталі будуть надіслані пізніше.

Рисунок 2.14 - Пункт меню “2” - “Показати повідомлення”

2.5.3 Третій пункт меню “Зберегти переписку”. При виборі пункта “3” всі повідомлення, які є в контейнері на даний момент збережуться (див. рис 2.15) в окремий текстовий файл з назвою “messages” (див. рис 2.16)

2.5.4 Четвертий пункт меню “Завантажити переписку”. При виборі пункта “4” всі повідомлення які є в текстовому файлі, завантажуються у програму (див. рис. 2.17) і зберігаються у контейнері, з якими надалі можна виконувати операції.

2.5.5 П’ятий пункт меню “Редагувати повідомлення” надає користувачу можливість змінити вже існуюче повідомлення за введеним ID. Після вибору цього пункту програма запитує ідентифікатор повідомлення, яке потрібно змінити. Якщо ID коректний, відкривається вікно редагування з підказкою щодо можливості використання форматування: жирний та курсив.

Користувач вводить новий текст повідомлення, використовуючи багаторядкове введення. Для завершення редагування необхідно ввести символ ::end на новому рядку. Весь процес супроводжується зрозумілим інтерфейсом і підказками (див. рис. 2.18):

МЕНЮ	
1	Додати повідомлення
2	Показати всі повідомлення
3	Зберегти переписку
4	Завантажити переписку
5	Редагувати повідомлення
6	Очистити переписку
7	Пошук повідомлення
8	Видалити повідомлення
9	Статистика чату
0	Вихід
Переписка збережена!	

Рисунок 2.15 - Пункт меню “3” - “Зберегти переписку”

```

ID: 1|тестове повідомлення\n*жирне*
\n_курсив_
-----
ID: 2|Нове\nповідомлення\n
-----
ID: 3|\n*Термінове повідомлення!*
\nСервер впав о 14:03\n_Причина
встановлюється_
-----
ID: 4|\nПривіт усім!\nЗустріч
переноситься на *п'ятницю*.\n_Деталі
будуть надіслані пізніше._
-----
ID: 5|Нагадування:\n- _Заповнити форму_
\n- *Надіслати на пошту*\n- Перевірити
статус
-----
ID: 6|_Короткий огляд змін:_\n* Додано
нову функцію пошуку\n* Виправлено
помилку з догуванням\n* Оптимізовано
завантаження
-----

```

Рисунок 2.16 - Збережена переписка у текстовому файлі

МЕНЮ	
1	Додати повідомлення
2	Показати всі повідомлення
3	Зберегти переписку
4	Завантажити переписку
5	Редагувати повідомлення
6	Очистити переписку
7	Пошук повідомлення
8	Видалити повідомлення
9	Статистика чату
0	Вихід
Переписка завантажена!	

Рисунок 2.17 - Пункт меню “4” - “Завантажити переписку”

```

-----
Підказка:
-----
Щоб зробити жирний або курсив
скористуйтеся форматуванням
*Жирний* _Курсив_
Щоб завершити введення, впишіть
/0 на новому рядку
-----
Введіть текст нового повідомлення: Нове
*тестове* _повідомлення_
(редагування *першого*)
/0|

```

Рисунок 2.18 - Пункт меню “5” - “Редагувати повідомлення”

Якщо користувач вводить некоректний або неіснуючий ID під час спроби редагування повідомлення, програма виконує перевірку введеного значення. У разі помилки користувача повідомляють про некоректність введених даних. Такий механізм запобігає спробам змінити повідомлення, яких не існує у сховищі(див.рис. 2.19-2.21).

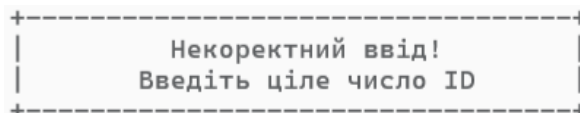


Рисунок 2.19 - Спроба ввести некоректний айді

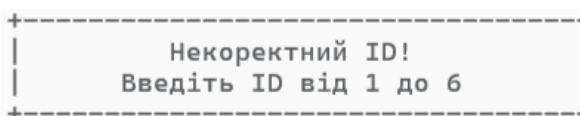


Рисунок 2.20 - Спроба ввести некоректний айді

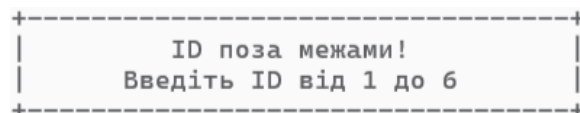


Рисунок 2.21 - Спроба ввести айді якого не існує

2.5.6 Шостий пункт меню “Очистити переписку” дозволяє повністю видалити всі наявні повідомлення з чату. Після вибору цього пункту (натискання клавіші “6”) програма запитує підтвердження дії з боку користувача, щоб уникнути випадкового видалення важливої інформації. (див. рис. 2.22).

У випадку підтвердження (y) викликається метод clearMessages(), який очищає контейнер повідомлень. Якщо введено будь-яке інше значення або “n”, дія скасовується і програма повертається до головного меню без змін.

МЕНЮ	
1	Додати повідомлення
2	Показати всі повідомлення
3	Зберегти переписку
4	Завантажити переписку
5	Редагувати повідомлення
6	Очистити переписку
7	Пошук повідомлення
8	Видалити повідомлення
9	Статистика чату
0	Вихід

Ви впевнені, що хочете видалити всі повідомлення? (y/n): y|

Рисунок 2.22 - Пункт меню “6” - “Очистити переписку”

2.5.7 Сьомий пункт меню “Пошук повідомлення” реалізує функціональність пошуку повідомлень за ключовим словом. Після вибору цього пункту (натискання клавіші “7”) програма запрошує користувача ввести слово або фрагмент тексту для пошуку. (див. рис. 2.23).

Результати пошуку
ID: 4 - Привіт усім! Зустріч переноситься на п'ятницю. Деталі будуть надіслані пізніше.

Рисунок 2.23 - Пункт меню “7” - “Пошук повідомлення”

Якщо користувач вводить коректне слово для пошуку, але жодне з повідомлень у чаті не містить цього слова, програма повідомляє, що відповідних результатів не знайдено. У такому випадку виводиться повідомлення (див. рис. 2.24).

Повідомлення не знайдено

Рисунок 2.24 - Повідомлення про те, що повідомлення не знайдено

2.5.8 Восьмий пункт меню “Видалити повідомлення” дозволяє користувачеві видалити окреме повідомлення з чату за його унікальним ID. Після вибору цього пункту (натискання клавіші “8”) програма запрошує ввести ID повідомлення, яке потрібно видалити (див. рис. 2.25).

МЕНЮ	
1	Додати повідомлення
2	Показати всі повідомлення
3	Зберегти переписку
4	Завантажити переписку
5	Редагувати повідомлення
6	Очистити переписку
7	Пошук повідомлення
8	Видалити повідомлення
9	Статистика чату
0	Вихід

Введіть ID повідомлення для видалення: 1
 Ви впевнені, що хочете видалити повідомлення з ID: 1? (y/n): y|

Рисунок 2.25 - Пункт меню “8” - “Видалення повідомлення”

Якщо користувач вводить коректний за форматом ID повідомлення, але повідомлення з таким ID не існує в контейнері, програма повідомляє про це відповідним повідомленням. Це дає змогу уникнути помилок при видаленні та інформує користувача про відсутність об'єкта з вказаним ідентифікатором (див. рис. 2.26-2.27).

Цей механізм перевірки гарантує, що видалення виконується лише для існуючих повідомлень, та забезпечує зворотний зв'язок у разі спроби взаємодії з неіснуючим елементом.

Некоректний ID! Введіть ID від 1 до 6
--

Рисунок 2.26 - Повідомлення про те, що повідомлення з таким айді немає

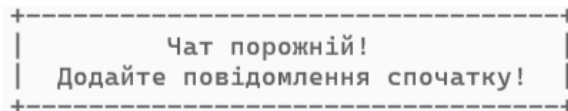


Рисунок 2.27 - Повідомлення якщо немає повідомлень для видалення

2.5.9 Дев'ятий пункт меню “Статистика чату”. При виборі пункта “9” програма показує повну статистику чату, в якій перелічено такі пункти:

“Всього повідомлень”, “Загальна кількість слів”, “Фрагментів жирного”, “Загальна кількість символів” (див.рис. 2.28).

Статистика чату	
Всього повідомлень	5
Загальна кількість слів	61
Фрагментів жирного	5
Фрагментів курсиву	5
Загальна кількість символів	462

Рисунок 2.28 - Пункт меню “9” - “Статистика чату”

2.5.10 Десятий пункт меню “Вихід”. Позначається “0”, при виборі цього пункту робота програми завершується та пропонує зберегти переписку (див. рис. 2.29).

МЕНЮ	
1	Додати повідомлення
2	Показати всі повідомлення
3	Зберегти переписку
4	Завантажити переписку
5	Редагувати повідомлення
6	Очистити переписку
7	Пошук повідомлення
8	Видалити повідомлення
9	Статистика чату
0	Вихід
Переписка збережена!	
Вихід...	

Рисунок 2.29 - Пункт меню “0” - “Вихід з програми”

2.6 Вимоги до складу та параметрів технічних та програмних засобів

Для повноцінного функціонування програми, необхідний персональний комп'ютер з такими характеристиками та програмним забезпеченням:

- операційна системи Windows, а саме Windows 10/11;
- оперативна пам'ять (RAM). Мінімальний обсяг оперативної пам'яті повинен бути не менше 4 ГБ для плавної роботи програми;
- для зберігання програмного забезпечення та даних може знадобитися достатньо великий дисковий простір, залежно від обсягу інформації, що обробляється, тому обсяг пам'яті в дисковому просторі повинен бути не менше 2 ГБ для потреб операційної системи;
- апаратне забезпечення (будь-які: клавіатура, миша), монітор (робочий) - з мінімальною роздільною здатністю - 800 x 600px.

2.7 Інструкція користувача програми

Для запуску програми необхідно відкрити проєкт у середовищі Visual Studio та запустити його натисканням клавіші Start. Після запуску на екрані з'являється головне меню, де користувач обирає одну з дій (див. рис. 2.30).

МЕНЮ	
1	Додати повідомлення
2	Показати всі повідомлення
3	Зберегти переписку
4	Завантажити переписку
5	Редагувати повідомлення
6	Очистити переписку
7	Пошук повідомлення
8	Видалити повідомлення
9	Статистика чату
0	Вихід

Виберіть дію: |

Рисунок 2.30 - Головне меню програми

Змін	Арк	№ докум	Підпис	Дата

ФКЗЕ.121ООП11.КРПЗ

Арк
45

2.7.1 Пункт меню 1 – “Додати повідомлення”.

Після вибору пункту 1 відкривається режим багаторядкового введення повідомлення. На екрані відображається підказка щодо використання форматування:

- ***текст*** - жирний;

- текст - курсив.

Завершення введення здійснюється введенням /0 на новому рядку. Повідомлення автоматично отримує унікальний ID. Якщо кількість символів форматування є непарною - відображається попередження (див. рис. 2.31).

```
+-----+
|               Підказка:               |
+-----+
|  Щоб зробити жирний або курсив       |
|  скористуйтеся форматуванням         |
|  *Жирний*  _Курсив_                  |
|  Щоб завершити введення, впишіть     |
|  /0 на новому рядку                  |
+-----+
Введіть текст повідомлення: тестове повідомлення
*жирне*
_курсив_
/0
```

Рисунок 2.31 - Додавання нового повідомлення з форматуванням

2.7.2 Пункт меню 2 – “Показати всі повідомлення”.

Програма відображає всі збережені повідомлення у табличному вигляді із зазначенням їх ID. Якщо чат порожній - з'являється відповідне повідомлення (див. рис. 2.32).

```
+-----+
|               Історія чату              |
+-----+
ID: 1 - тестове повідомлення
жирне
курсив

ID: 2 - Звіт про помилку:
1. Не відкривається файл
2. Виводиться повідомлення про помилку
Рішення: перевірити права доступу

ID: 3 -
Термінове повідомлення!
Сервер впав о 14:03
Причина встановлюється
```

Рисунок 2.32 - Вивід усіх повідомлень про порожній чат

2.7.3 Пункт меню 3 – “Зберегти переписку”.

Цей пункт зберігає всі повідомлення у файл messages.txt. Форматування зберігається шляхом перетворення переносу рядків у символ \n (див. рис. 2.33, 2.34).

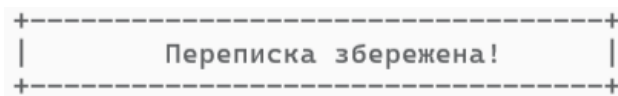


Рисунок 2.33 - Успішне збереження чату до файлу

```
ID: 1|тестове повідомлення\n*жирне*\n_курсив_\n-----\nID: 2|Нове\n_повідомлення\n-----\nID: 3|\n*Термінове повідомлення!*\n_Сервер впав о 14:03\n_Причина встановлюється_\n-----\nID: 4|\nПривіт усім!\n_зустріч переноситься на *п'ятницю*.\n_Деталі будуть надіслані пізніше._\n-----\nID: 5|Нагадування:\n_Заповнити форму_\n_Надіслати на пошту*\n_Перевірити статус_\n-----\nID: 6|_Короткий огляд змін:\n_Додано нову функцію пошуку\n_Виправлено помилку з догуванням\n_Оптимізовано завантаження_\n-----
```

Рисунок 2.34 - Збережена переписка у текстовому файлі

2.7.4 Пункт меню 4 – “Завантажити переписку”.

Після вибору цього пункту програма зчитує файл messages.txt, розпізнає ID та текст, відновлює багаторядковість і відображає результат. Якщо файл порожній або відсутній - з'явиться відповідне повідомлення (див. рис. 2.35).



Рисунок 2.35 - Завантаження повідомлень з файлу

2.7.5 Пункт меню 5 – “Редагувати повідомлення”.

Користувач вводить ID повідомлення для редагування. Програма виконує перевірку на коректність ID та існування повідомлення. Далі відкривається багаторядкове поле для введення нового тексту, аналогічне пункту 1. Після обрання пункту збереження даних відбувається оновлення старого тексту на новий (див. рис. 2.36).

2.7.6 Пункт меню 6 – “Очистити переписку”.

Після вибору цього пункту користувачу потрібно підтвердити дію, ввівши Y або N. Якщо підтвердження надано - чат повністю очищується (див. рис. 2.37).

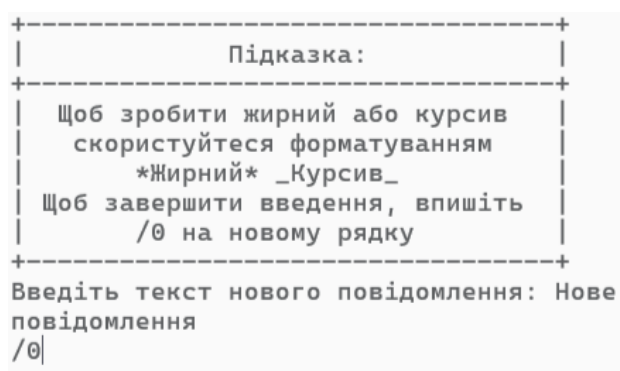


Рисунок 2.36 - Редагування повідомлення за ID

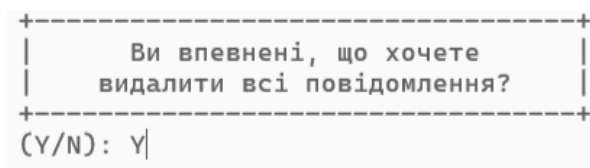


Рисунок 2.37 - Підтвердження очищення чату

2.7.7 Пункт меню 7 – “Пошук повідомлення”.

Після введення ключового слова, програма здійснює пошук у всіх повідомленнях. Якщо знайдені результати - вони виводяться на екран, інакше з’являється повідомлення про відсутність збігів (див. рис. 2.38).

```

+-----+
|           Результати пошуку           |
+-----+
ID: 4 -
Привіт усім!
Зустріч переноситься на п'ятницю.
Деталі будуть надіслані пізніше.
Виберіть дію: |

```

Рисунок 2.38 - Результати пошуку повідомлень

2.7.8 Пункт меню 8 – “Видалити повідомлення”.

Користувач вводить ID повідомлення, яке потрібно видалити. Програма перевіряє існування ID і просить підтвердження. Після підтвердження повідомлення видаляється з пам'яті (див. рис. 2.39).

```

+-----+
|           Ви впевнені, що хочете       |
|           видалити повідомлення з ID: 1? |
+-----+
(Y/N): Y|

```

Рисунок 2.39 - Видалення повідомлення з підтвердженням

2.7.9 Пункт меню 9 – “Статистика чату”.

Програма виводить статистичні дані:

- кількість повідомлень;
- кількість слів та символів;
- кількість жирних та курсивних фрагментів (визначаються за * та _);
- середню довжину повідомлення (див. рис. 2.40).

2.7.10 Пункт меню 0 – “Вихід з програми”.

При завершенні роботи користувач отримує запит, чи потрібно зберегти переписку. У разі підтвердження викликається процедура збереження. Після завершення програма закривається (див. рис. 2.41).

Статистика чату	
Всього повідомлень	6
Загальна кількість слів	51
Фрагментів жирного	5
Фрагментів курсиву	5
Загальна кількість символів	398

Рисунок 2.40 - Вивід статистики чату

```

+-----+
|               Переписка збережена!               |
+-----+
Вихід...

```

Рисунок 2.41 - Завершення роботи програми з збереженням

2.8 Результати реалізації програми

У прикладі розробленої ієрархії класів - Message, SimpleMessage, MessageDecorator, BoldMessageDecorator, ItalicMessageDecorator, MessageStorage - були успішно застосовані концепції об'єктно-орієнтованого програмування (наслідування, поліморфізм, інкапсуляція) та використано засоби стандартної бібліотеки шаблонів (STL), зокрема контейнер set, shared_ptr для управління пам'яттю та алгоритми для пошуку й обробки тексту.

Клас MessageStorage відповідає за управління повідомленнями: збереження у файл, пошук, редагування та видалення. Форматування (жирний, курсив) реалізовано через шаблон "Декоратор", що дозволяє додавати функціонал без змін базового класу.

Програма підтримує кирилицю й латиницю, коректно обробляє переноси рядків (\n) при збереженні та завантаженні, зберігаючи багаторядкову структуру.

Передбачено перевірку введення, підтвердження дій, кольоровий інтерфейс, текстові підказки в меню та обмеження на службові символи (наприклад, |).

Пропозиції щодо усунення недоліків та вдосконалення. Для усунення недоліків і подальшого вдосконалення програми можна запропонувати такі зміни:

Розширення валідації вводу: запровадити додаткові перевірки - наприклад, дозволити у повідомленнях лише літери, цифри та базові символи пунктуації, або запровадити режим фільтрації за мовою/алфавітом.

Пошук із урахуванням регістру та розширена фільтрація: додати можливість пошуку незалежно від регістру (upper/lower case) або з додатковими фільтрами (наприклад, лише жирні повідомлення, лише за датою, тощо).

					ФКЗЕ.121ООП11.КРПЗ	Арк
						51
Змін	Арк	№ докум	Підпис	Дата		

ВИСНОВКИ ПО РОБОТІ

У даній курсовій роботі було реалізовано ієрархію класів відповідно до постановки задачі: Message, SimpleMessage, MessageDecorator, BoldMessageDecorator, ItalicMessageDecorator, MessageStorage, а також створено консольну програму для демонстрації можливостей роботи з повідомленнями. Було визначено необхідні методи (конструктори, гетери, сетери, методи обробки даних), а також реалізовано збереження, редагування, пошук і видалення повідомлень із застосуванням форматування.

Використання контейнера set та інструментів STL дало змогу удосконалити практичні навички роботи з динамічними структурами даних, ітераторами, потоками вводу/виводу (cin, cout, fstream), а також методами опрацювання рядкових даних.

У процесі виконання курсової роботи було поглиблено знання з ключових концепцій об'єктно-орієнтованого програмування: створення класів, наслідування, поліморфізм, використання абстрактних класів і шаблонів проектування. Окрему увагу приділено вивченню та застосуванню патерна “Декоратор”, що дозволив реалізувати гнучке форматування повідомлень без зміни їх базової структури.

ПЕРЕЛІК ВИКОРИСТАНИХ ДЖЕРЕЛ

- 1 Бублик В.В. Об'єктно-орієнтоване програмування: Підручник. Київ: ІТ-книга, 2015. - 624 с.: іл.
- 2 Грицюк Ю. І., Рак Т. Є. Об'єктно-орієнтоване програмування мовою C++ : навч. посібник. - Львів : Вид-во Львівського ДУБЖД, 2011. - 404 с
- 3 Lospinoso, Josh. C++ Crash Course: A Fast-Paced Introduction. San Francisco: No Starch Press, 2019.
- 4 Табунщик, Г. В. Проектування та моделювання програмного забезпечення сучасних інформаційних систем / Г. В. Табунщик, Т.І. Каплієнко, О.А. Петрова – Запоріжжя : Дике Поле, 2016. - 250 с.
- 5 Введення у `std::vector`. URL: https://hackingcpp.com/cpp/std/associative_containers.html#set (Дата звернення 28.03.2025).
- 6 Поведінковий патерн Декоратор. URL: <https://refactoring.guru/uk/design-patterns/decorator> (Дата звернення 28.03.2025).
- 7 ChatGPT URL: <https://chatgpt.com/>.

Додаток А

Блок-схеми

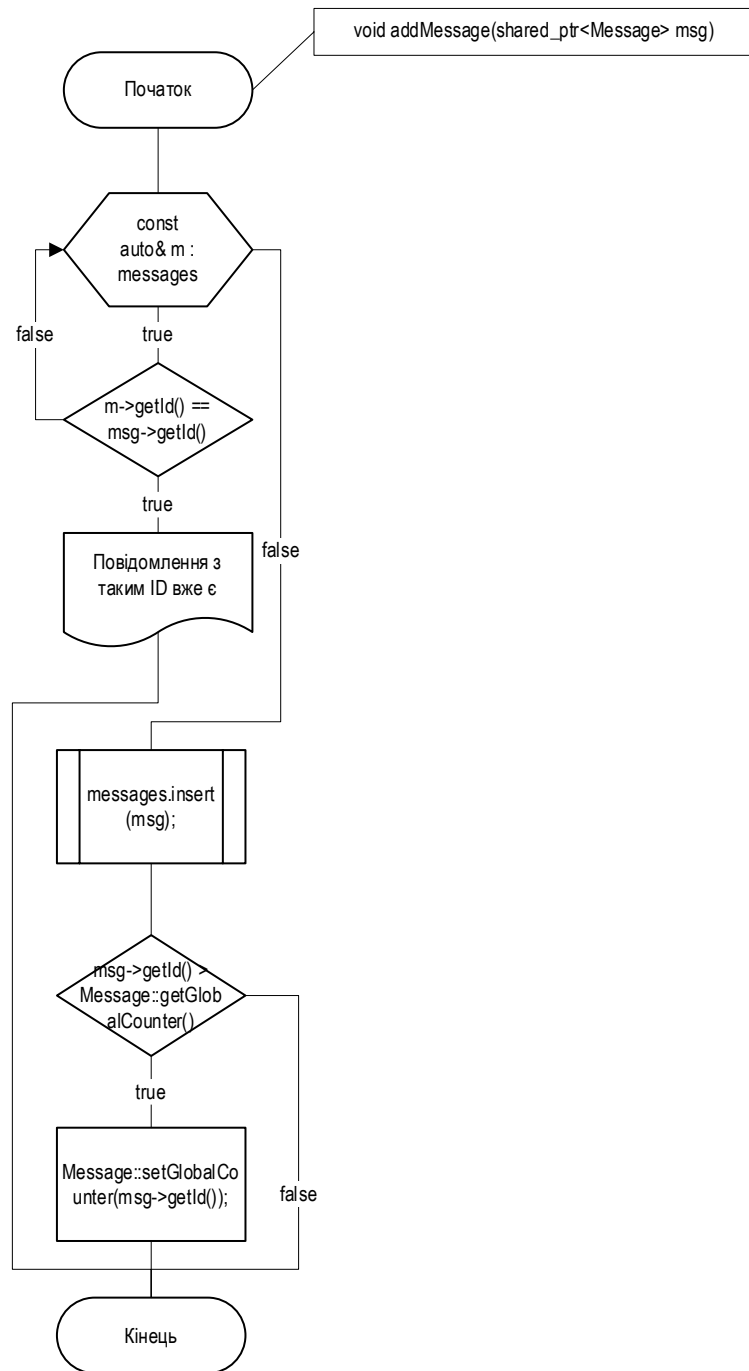


Рисунок А.1 - Демонстрація алгоритму додавання повідомлень до контейнеру

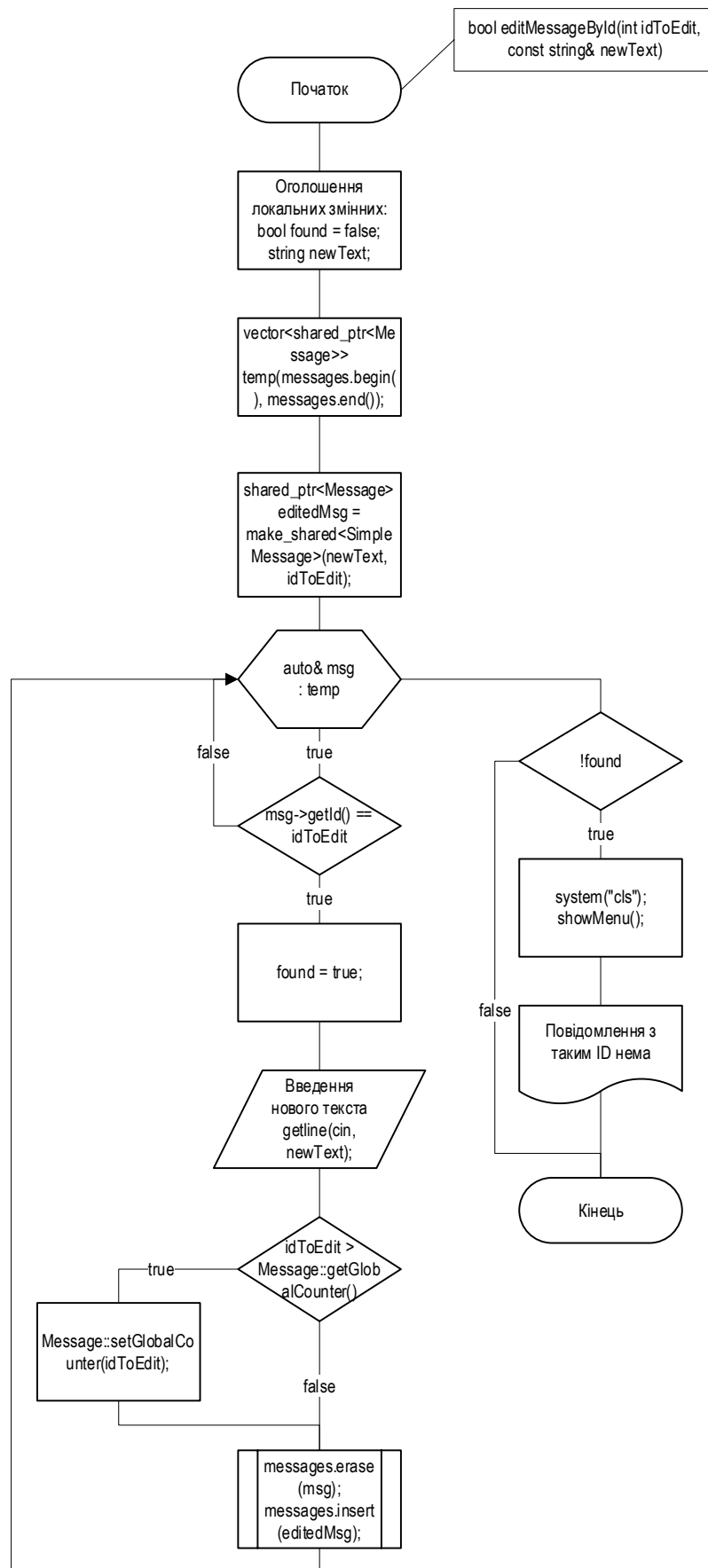


Рисунок А.2 - Демонстрація алгоритму редагування повідомлень

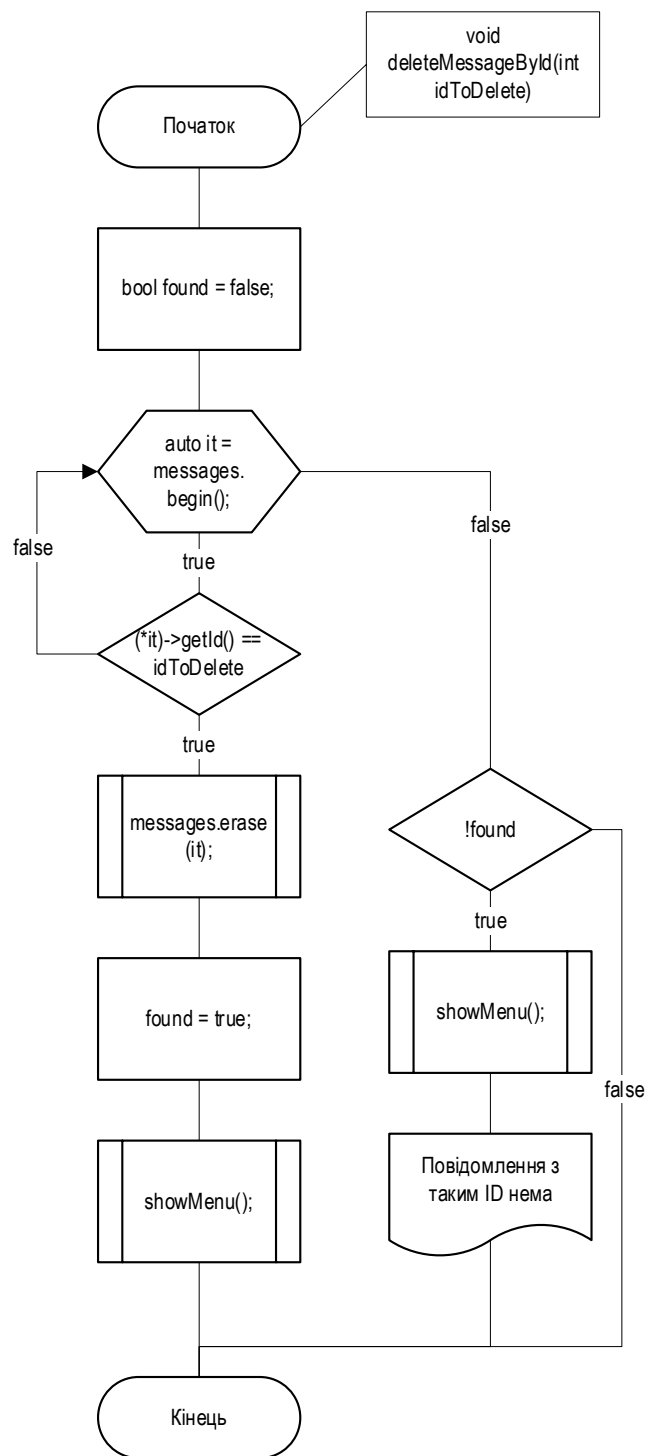


Рисунок А.3 - Демонстрація алгоритму видалення повідомлень

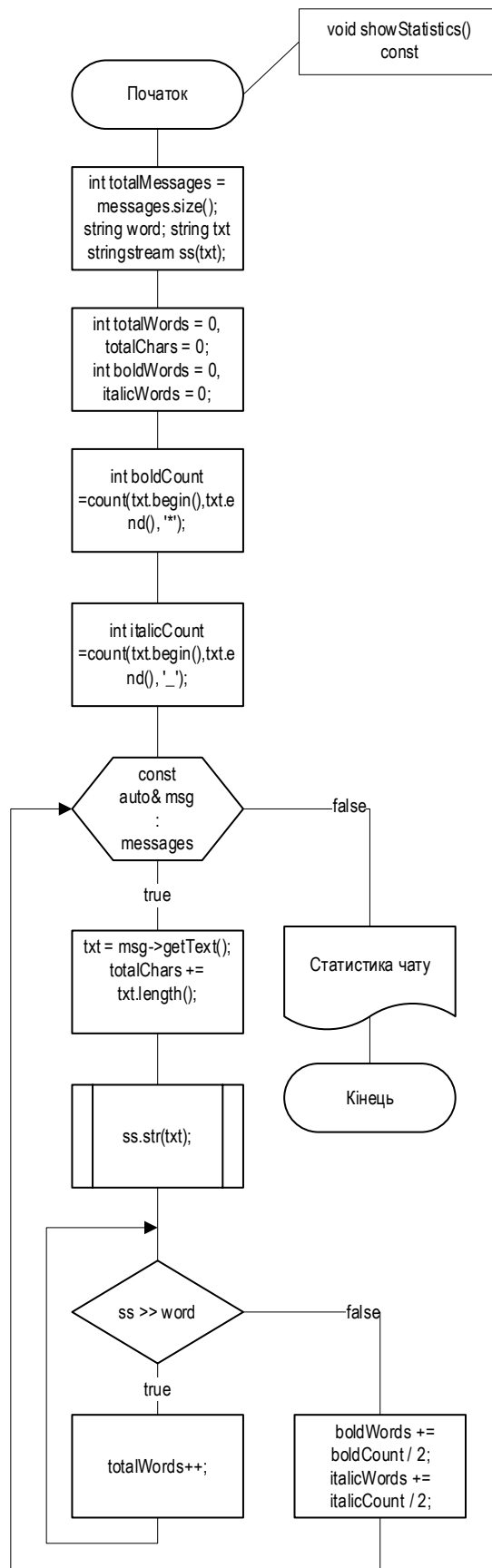


Рисунок А.4 - Демонстрація алгоритму виводу статистики повідомлень

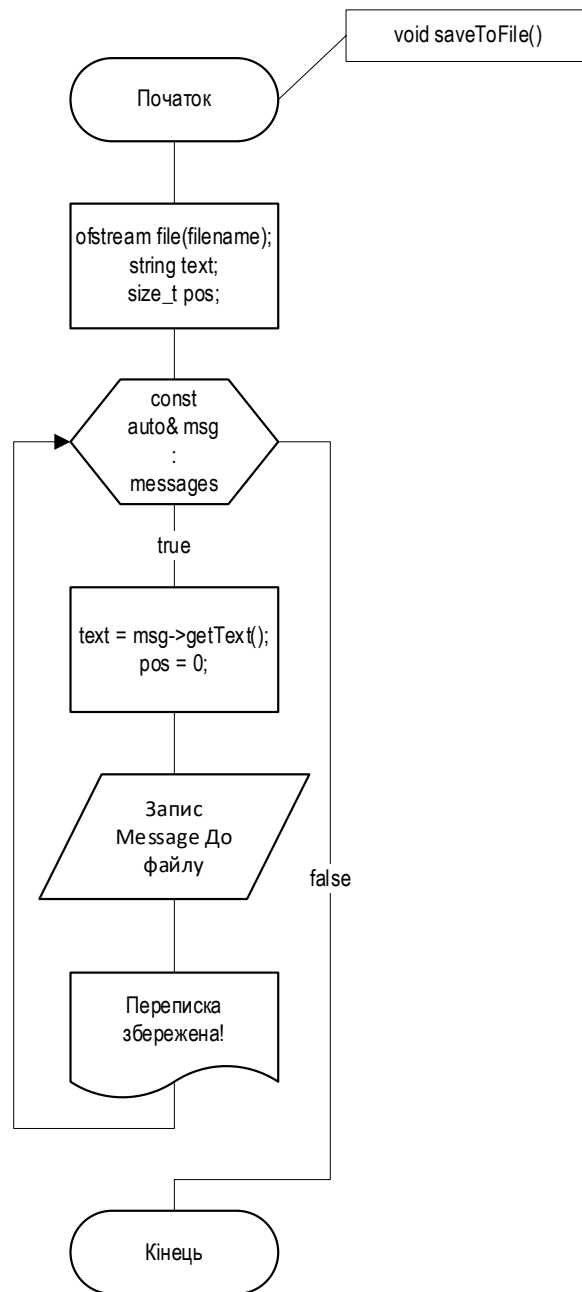


Рисунок А.5 - Демонстрація алгоритму збереження переписки до текстового файлу

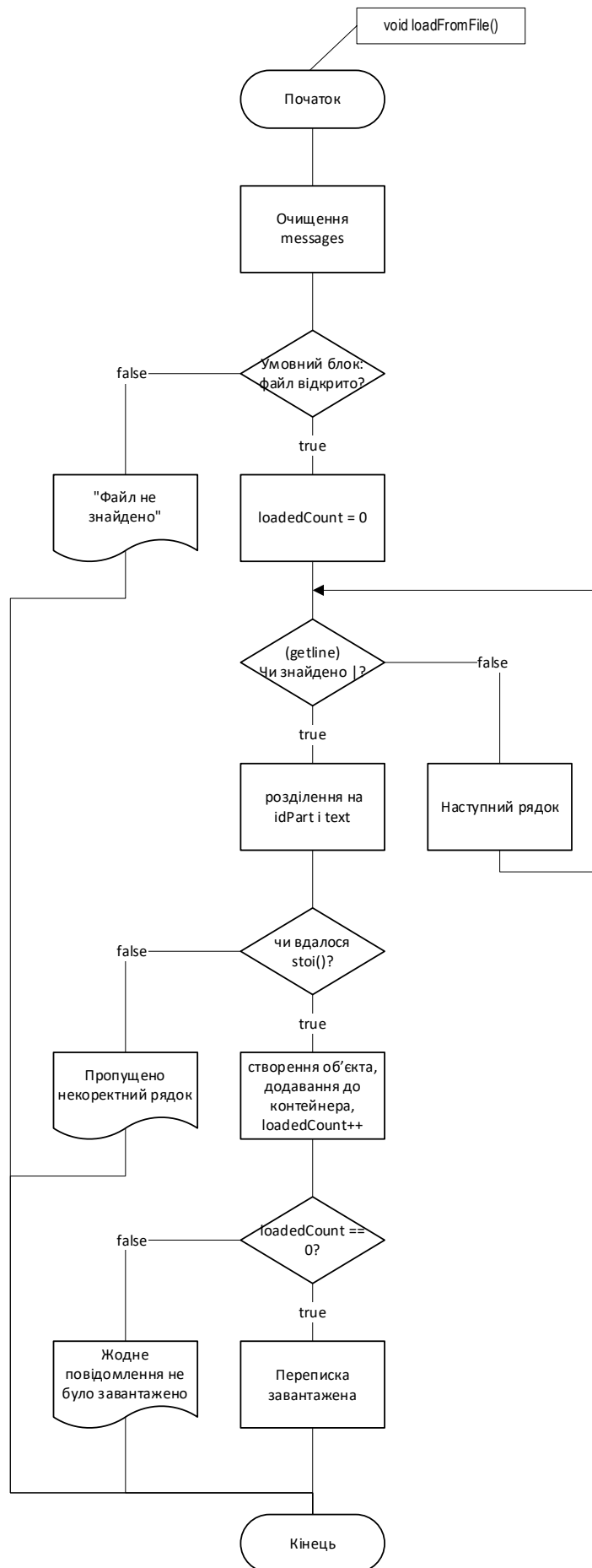


Рисунок А.6 - Демонстрація алгоритму завантаження з файлу

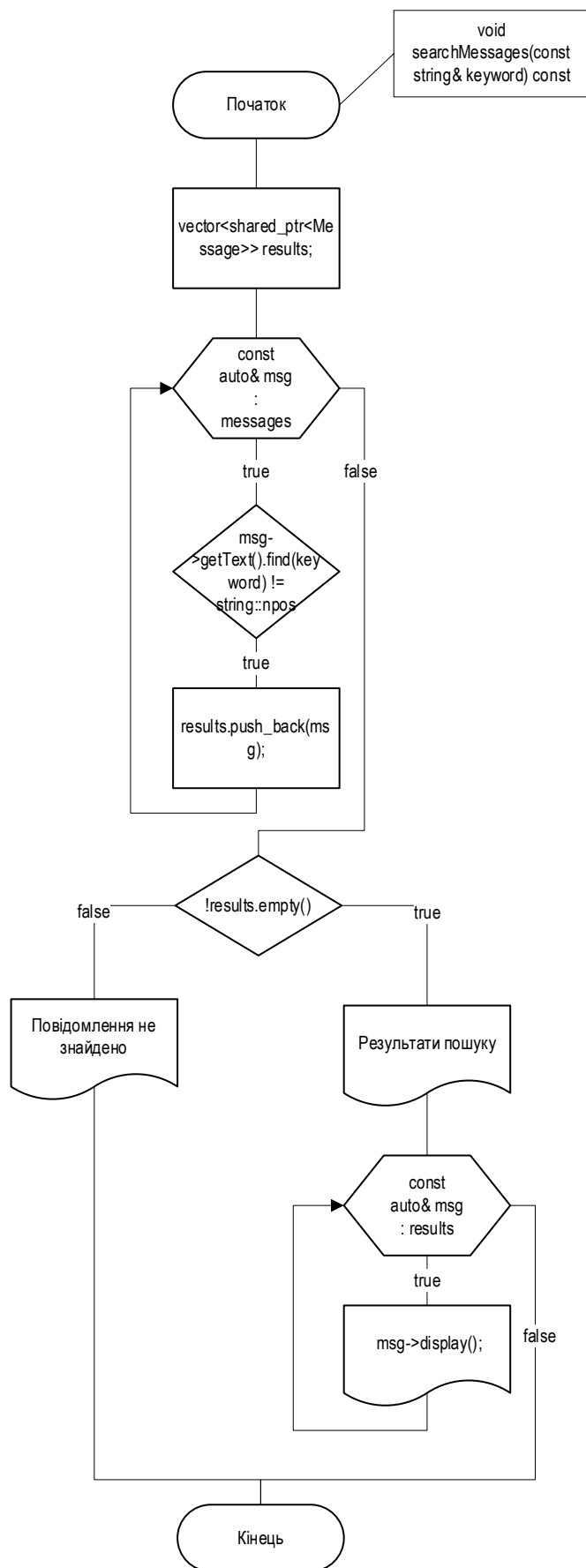


Рисунок А.7 - Демонстрація алгоритму пошуку повідомлень

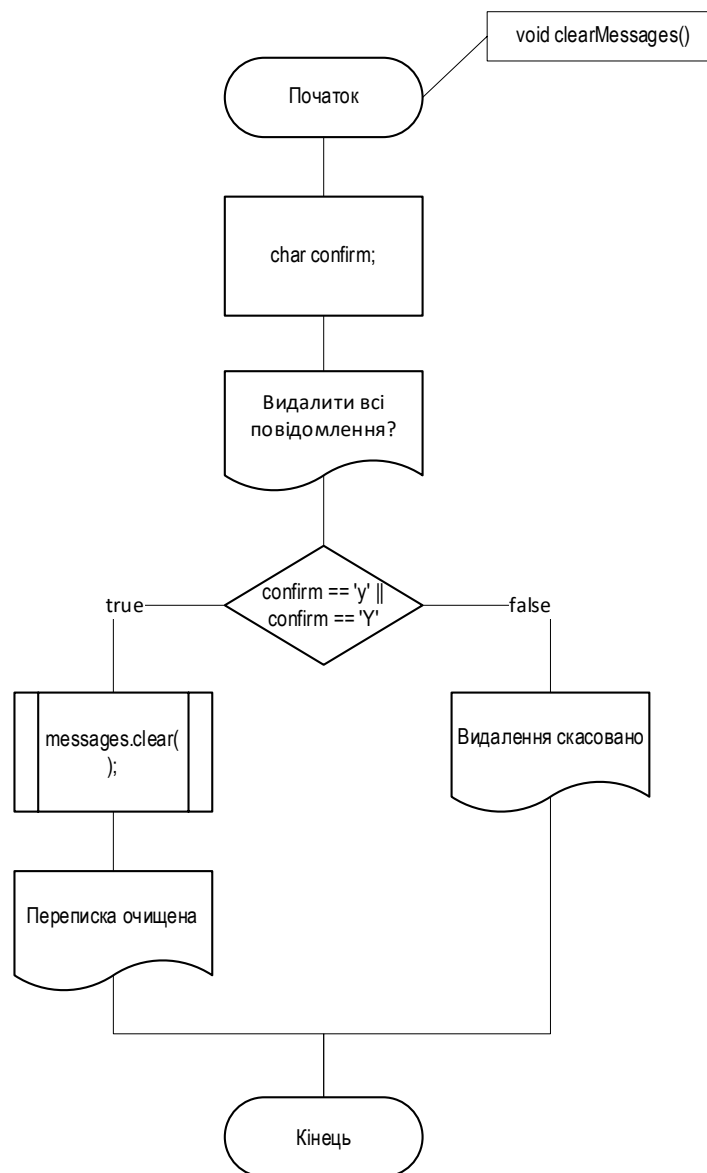


Рисунок А.8 - Демонстрація алгоритму очищення повідомлень

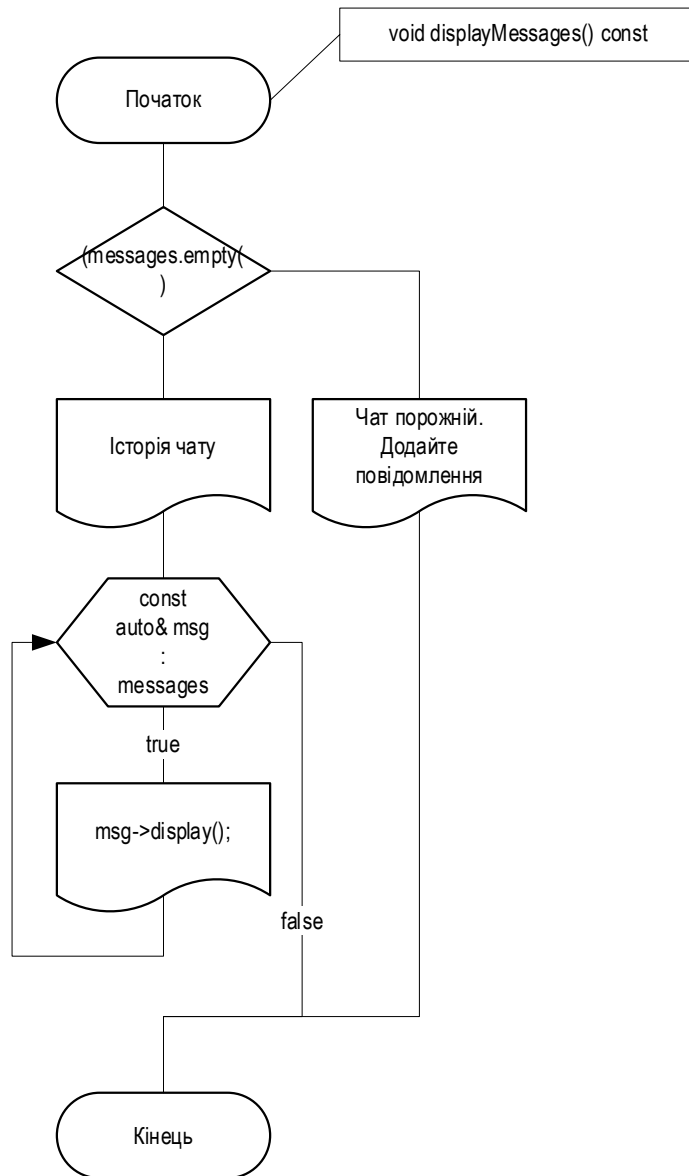


Рисунок А.9 - Демонстрація алгоритму виводу повідомлень

Додаток Б

Лістинг програми

```
////////// MessageApp.cpp //////////

#include <iostream>
#include <iomanip>
#include <set>
#include <fstream>
#include <sstream>
#include <memory>
#include <windows.h>
#include <vector>

using namespace std;

void setConsoleColor(WORD color) {
    SetConsoleTextAttribute(GetStdHandle(STD_OUTPUT_HANDLE), color);
}

void showMenu() {
    cout << "+-----+" << endl;
    cout << "|                МЕНЮ                |" << endl;
    cout << "+-----+" << endl;
    cout << "| 1 | Додати повідомлення |" << endl;
    cout << "| 2 | Показати всі повідомлення |" << endl;
    cout << "| 3 | Зберегти переписку |" << endl;
    cout << "| 4 | Завантажити переписку |" << endl;
    cout << "| 5 | Редагувати повідомлення |" << endl;
    cout << "| 6 | Очистити переписку |" << endl;
    cout << "| 7 | Пошук повідомлення |" << endl;
    cout << "| 8 | Видалити повідомлення |" << endl;
    cout << "| 9 | Статистика чату |" << endl;
    cout << "| 0 | Вихід |" << endl;
    cout << "+-----+" << endl;
}

class Message {
protected:
    static int global_id_counter;
    int id;
    string text;

public:
    Message(const string& txt)
        : text(txt), id(++global_id_counter) {}

    Message(const string& txt, int forcedId)
        : text(txt), id(forcedId)
    {
        if (forcedId > global_id_counter) {
            global_id_counter = forcedId;
        }
    }

    virtual ~Message() {}

    void setId(int newId) { id = newId; }
```

```

    int getId() const { return id; }

    static int getGlobalCounter() { return global_id_counter; }
    static void setGlobalCounter(int value) { global_id_counter = value; }

    virtual string getText() const {
        return text;
    }

    virtual string getType() const {
        return "Просьба";
    }

    virtual void display() const {
        cout << "ID: " << id << " - " << getText() << endl;
    }

    bool operator<(const Message& other) const {
        return id < other.id;
    }
};

int Message::global_id_counter = 0;

class SimpleMessage : public Message {
public:
    SimpleMessage(const string& txt)
        : Message(txt) {}

    SimpleMessage(const string& txt, int forcedId)
        : Message(txt, forcedId) {}

    void display() const override {
        setConsoleColor(8);
        cout << "ID: " << getId() << " - ";
        string rawText = this->getText();
        applyFormatting(rawText);
        setConsoleColor(8);
    }

protected:
    void applyFormatting(const string& text) const {
        HANDLE hConsole = GetStdHandle(STD_OUTPUT_HANDLE);
        CONSOLE_SCREEN_BUFFER_INFO csbi;
        GetConsoleScreenBufferInfo(hConsole, &csbi);
        WORD defaultColor = csbi.wAttributes;

        bool bold = false, italic = false;

        for (size_t i = 0; i < text.length(); i++) {
            if (text[i] == '*' && (i == 0 || text[i - 1] != '\\')) {
                bold = !bold;
                setConsoleColor(bold ? 0 : defaultColor);
                continue;
            }
            if (text[i] == '_' && (i == 0 || text[i - 1] != '\\')) {
                italic = !italic;
                cout << (italic ? "\033[3m" : "\033[0m");
                if (!italic && bold) setConsoleColor(0);
                else if (!italic) setConsoleColor(defaultColor);
                continue;
            }
            cout << text[i];
        }
    }
};

```

```

        setConsoleColor(defaultColor);
        cout << "\033[0m" << endl;
    }
};

class MessageDecorator : public Message {
protected:
    shared_ptr<Message> wrappedMessage;

public:
    MessageDecorator(shared_ptr<Message> msg)
        : Message(msg->getText(), msg->getId()), wrappedMessage(msg) {}

    string getText() const override {
        return wrappedMessage->getText();
    }

    virtual void display() const override {
        wrappedMessage->display();
    }
};

class BoldMessageDecorator : public MessageDecorator {
public:
    BoldMessageDecorator(shared_ptr<Message> msg) : MessageDecorator(msg) {}

    string getType() const override {
        return "Bold";
    }

    void display() const override {
        setConsoleColor(0xF0);
        cout << "ID: " << getId() << " - " << getText() << endl;
        setConsoleColor(8);
    }
};

class ItalicMessageDecorator : public MessageDecorator {
public:
    ItalicMessageDecorator(shared_ptr<Message> msg) : MessageDecorator(msg) {}

    string getType() const override {
        return "Italic";
    }

    void display() const override {
        setConsoleColor(8);
        cout << "ID: " << getId() << " - " << "\033[3m" << getText() << "\033[0m" <<
endl;
        setConsoleColor(8);
    }
};

struct MessageComparator {
    bool operator()(const shared_ptr<Message>& lhs, const shared_ptr<Message>& rhs)
const {
        return lhs->getId() < rhs->getId();
    }
};

```

```

class MessageStorage {
private:
    set<shared_ptr<Message>, MessageComparator> messages;
    string filename = "messages.txt";

public:
    const set<shared_ptr<Message>, MessageComparator>& getMessages() const {
        return messages;
    }

    void addMessage(shared_ptr<Message> msg) {
        for (const auto& m : messages) {
            if (m->getId() == msg->getId()) {
                cout << "|    Повідомлення з таким ID вже є    |" << endl;
                cout << "+-----+" << endl;
                return;
            }
        }
        messages.insert(msg);

        // Після додавання оновлюємо глобальний лічильник
        if (msg->getId() > Message::getGlobalCounter()) {
            Message::setGlobalCounter(msg->getId());
        }
    }

    void displayMessages() const {
        if (messages.empty()) {
            cout << "|          Чат порожній.          |" << endl;
            cout << "|    Додайте повідомлення!    |" << endl;
            cout << "+-----+" << endl;
            return;
        }
        cout << "|          Історія чату          |" << endl;
        cout << "+-----+" << endl;
        for (const auto& msg : messages) {
            msg->display();
            cout << endl;
        }
    }

    bool editMessageById(int idToEdit, const string& newText) {
        for (auto& msg : messages) {
            if (msg->getId() == idToEdit) {
                shared_ptr<Message> editedMsg = make_shared<SimpleMessage>(newText,
idToEdit);

                messages.erase(msg);
                messages.insert(editedMsg);

                if (idToEdit > Message::getGlobalCounter()) {
                    Message::setGlobalCounter(idToEdit);
                }

                return true;
            }
        }
        return false;
    }

    void deleteMessageById(int idToDelete) {
        bool found = false;

```

```

        for (auto it = messages.begin(); it != messages.end(); ++it) {
            if ((*it)->getId() == idToDelete) {
                messages.erase(it);
                found = true;
                system("cls");
                showMenu();
                cout << "|    Повідомлення видалено успішно    |" << endl;
                cout << "+-----+" << endl;
                return;
            }
        }
        if (!found) {
            system("cls");
            showMenu();
            cout << "|    Повідомлення з таким ID нема    |" << endl;
            cout << "+-----+" << endl;
        }
    }

void showStatistics() const {
    int totalMessages = messages.size();
    int totalWords = 0, totalChars = 0;
    int boldWords = 0, italicWords = 0;

    for (const auto& msg : messages) {
        string txt = msg->getText();
        totalChars += txt.length();

        // Рахуємо слова
        stringstream ss(txt);
        string word;
        while (ss >> word) {
            totalWords++;
        }

        // Рахуємо * і _ по всьому тексту
        int boldCount = count(txt.begin(), txt.end(), '*');
        int italicCount = count(txt.begin(), txt.end(), '_');

        boldWords += boldCount / 2;
        italicWords += italicCount / 2;
    }

    cout << "|          Статистика чату          |" << endl;
    cout << "+-----+" << endl;
    cout << "| Всього повідомлень                " << setw(4) << totalMessages << " |"
<< endl;
    cout << "| Загальна кількість слів          " << setw(4) << totalWords << " |" <<
endl;
    cout << "| Фрагментів жирного                " << setw(4) << boldWords << " |" <<
endl;
    cout << "| Фрагментів курсиву                " << setw(4) << italicWords << " |" <<
endl;
    cout << "| Загальна кількість символів      " << setw(4) << totalChars << " |" <<
endl;
    cout << "+-----+" << endl;
}

void saveToFile() {

```

```

        ofstream file(filename);
        for (const auto& msg : messages) {
            string text = msg->getText();
            size_t pos = 0;
            while ((pos = text.find('\n', pos)) != string::npos) {
                text.replace(pos, 1, "\\n");
                pos += 2;
            }
            file << "ID: " << msg->getId() << "|" << text << endl;
            file << "-----" << endl;
        }
        system("cls");
        showMenu();
        cout << "|           Переписка збережена!           |" << endl;
        cout << "+-----+" << endl;
    }

    void loadFromFile() {
        messages.clear();
        ifstream file(filename);

        if (!file.is_open()) {
            system("cls");
            showMenu();
            cout << "|           Файл не знайдено!           |" << endl;
            cout << "+-----+" << endl;
            return;
        }

        string line;
        int loadedCount = 0;

        while (getline(file, line)) {
            size_t delim = line.find('|');
            if (delim != string::npos) {
                string idPart = line.substr(0, delim);
                string text = line.substr(delim + 1);
                size_t colon = idPart.find(':');

                if (colon != string::npos) {
                    string idStr = idPart.substr(colon + 1);
                    try {
                        int id = stoi(idStr);

                        // Зворотня заміна \\n на реальні переноси
                        size_t pos = 0;
                        while ((pos = text.find("\\n", pos)) != string::npos) {
                            text.replace(pos, 2, "\n");
                            pos += 1;
                        }

                        shared_ptr<Message> msg = make_shared<SimpleMessage>(text,
id);

                        messages.insert(msg);
                        loadedCount++;
                    }
                    catch (...) {
                        cout << "Пропущено некоректний рядок: " << line << endl;
                    }
                }
            }
        }

        if (loadedCount == 0) {

```



```

        system("cls");
        showMenu();
        cout << "|          Жодне повідомлення не було          |" << endl;
        cout << "|                                завантажено                                |" << endl;
        cout << "+-----+" << endl;
    }
    else {
        system("cls");
        showMenu();
        cout << "|          Переписка завантажена!          |" << endl;
        cout << "+-----+" << endl;
    }
}

void searchMessages(const string& keyword) const {
    vector<shared_ptr<Message>> results;

    for (const auto& msg : messages) {
        if (msg->getText().find(keyword) != string::npos) {
            results.push_back(msg);
        }
    }

    if (!results.empty()) {
        system("cls");
        showMenu();
        cout << "|          Результати пошуку          |" << endl;
        cout << "+-----+" << endl;
        for (const auto& msg : results) {
            msg->display();
        }
    }
    else {
        system("cls");
        showMenu();
        cout << "|          Повідомлення не знайдено          |" << endl;
        cout << "+-----+" << endl;
    }
}

void clearMessages() {
    char confirm;
    system("cls");
    showMenu();
    cout << "|          Ви впевнені, що хочете          |" << endl;
    cout << "|          видалити всі повідомлення?          |" << endl;
    cout << "+-----+" << endl;
    cout << "(Y/N): ";
    cin >> confirm;
    cin.ignore(); // Очищення буфера

    if (confirm == 'y' || confirm == 'Y') {
        messages.clear();
        system("cls");
        showMenu();
        cout << "|          Переписка очищена          |" << endl;
        cout << "+-----+" << endl;
    }
    else {
        system("cls");
        showMenu();
        cout << "|          Видалення скасовано          |" << endl;
        cout << "+-----+" << endl;
    }
}

```

```

    }
}

};

bool isCancelled(const string& input) {
    return input == "/cancel";
}

void addMessageFlow(MessageStorage& storage) {
    system("cls");
    showMenu();
    cout << "|                Підказка:                |" << endl;
    cout << "+-----+" << endl;
    cout << "|  Щоб зробити жирний або курсив  |" << endl;
    cout << "|  скористуйтеся форматуванням    |" << endl;
    cout << "|          *Жирний* _Курсив_      |" << endl;
    cout << "|  Щоб завершити введення, впишіть |" << endl;
    cout << "|          /0 на новому рядку      |" << endl;
    cout << "|  АБО /cancel – щоб вийти без змін |" << endl;
    cout << "+-----+" << endl;

    cout << "Введіть текст повідомлення: ";

    string line, text;
    while (true) {
        getline(cin, line);
        if (isCancelled(line)) {
            system("cls");
            showMenu();
            cout << "|                Дію скасовано!                |" << endl;
            cout << "+-----+" << endl;
            return;
        }

        if (line == "/0") break;

        text += line + "\n";
    }

    if (text.empty()) {
        cout << "Повідомлення не може бути порожнім." << endl;
        return;
    }

    if (text.find('|') != string::npos) {
        cout << "Символ '|' заборонено!" << endl;
        return;
    }

    shared_ptr<Message> msg = make_shared<SimpleMessage>(text);

    int stars = count(text.begin(), text.end(), '*');
    int underscores = count(text.begin(), text.end(), '_');
    if (stars % 2 != 0) {
        system("cls");
        showMenu();
        cout << "|                Увага!                |" << endl;
        cout << "|                непарна кількість *      |" << endl;
        cout << "|                форматування може бути    |" << endl;
        cout << "|                некоректним!              |" << endl;
        cout << "+-----+" << endl;
    }
}

```

```

    }
    if (underscores % 2 != 0) {
        system("cls");
        showMenu();
        cout << "|                Увага!                |" << endl;
        cout << "|                непарна кількість _        |" << endl;
        cout << "|                форматування може бути      |" << endl;
        cout << "|                некоректним!                |" << endl;
        cout << "+-----+" << endl;
    }

    storage.addMessage(msg);
    system("cls");
    showMenu();
    cout << "|                Повідомлення додано!        |" << endl;
    cout << "+-----+" << endl;
}

void editMessageFlow(MessageStorage& storage) {
    system("cls");
    showMenu();

    if (storage.getMessages().empty()) {
        cout << "|                Чат порожній!                |" << endl;
        cout << "|                Додайте спочатку повідомлення |" << endl;
        cout << "+-----+" << endl;
        return;
    }

    // Підказка
    cout << "|                Підказка:                |" << endl;
    cout << "+-----+" << endl;
    cout << "|                Щоб зробити жирний або курсив |" << endl;
    cout << "|                скористуйтеся форматуванням   |" << endl;
    cout << "|                *Жирний* _Курсив_            |" << endl;
    cout << "|                Щоб завершити введення, впишіть |" << endl;
    cout << "|                /0 на новому рядку            |" << endl;
    cout << "|                АБО /cancel – щоб вийти без змін |" << endl;
    cout << "+-----+" << endl;

    // Введення ID
    string inputId;
    cout << "Введіть ID повідомлення для редагування: ";
    getline(cin, inputId);
    if (inputId == "/cancel") {
        system("cls");
        showMenu();
        cout << "|                Дію скасовано користувачем    |" << endl;
        cout << "+-----+" << endl;
        return;
    }

    int id;
    try {
        id = stoi(inputId);
    }
    catch (...) {
        system("cls");
        showMenu();
        cout << "|                Некоректний ввід!                |" << endl;
        cout << "|                Введіть ціле число ID          |" << endl;
        cout << "+-----+" << endl;
        return;
    }
}

```

```

// Пошук повідомлення з таким ID
shared_ptr<Message> originalMsg = nullptr;
for (const auto& msg : storage.getMessages()) {
    if (msg->getId() == id) {
        originalMsg = msg;
        break;
    }
}

if (!originalMsg) {
    system("cls");
    showMenu();
    cout << "|   Повідомлення з таким ID нема!   |" << endl;
    cout << "+-----+" << endl;
    return;
}

// Введення нового тексту
string line, newText;
cout << "Введіть новий текст повідомлення:";
while (true) {
    getline(cin, line);
    if (line == "/cancel") {
        system("cls");
        showMenu();
        cout << "|           Редагування скасовано!           |" << endl;
        cout << "+-----+" << endl;
        return;
    }
    if (line == "/0") break;
    newText += line + "\n";
}

// Перевірки
if (newText.empty()) {
    system("cls");
    showMenu();
    cout << "|           Повідомлення не може           |" << endl;
    cout << "|           бути порожнім!           |" << endl;
    cout << "+-----+" << endl;
    return;
}

if (newText.find('|') != string::npos) {
    system("cls");
    showMenu();
    cout << "|           Символ '|' заборонено!           |" << endl;
    cout << "+-----+" << endl;
    return;
}

// Створення нового повідомлення й оновлення
shared_ptr<Message> editedMsg = make_shared<SimpleMessage>(newText, id);
storage.deleteMessageById(id);
storage.addMessage(editedMsg);

system("cls");
showMenu();
cout << "|   Повідомлення відредаговано!   |" << endl;
cout << "+-----+" << endl;
}

```

```

void searchMessageFlow(MessageStorage& storage) {
    system("cls");
    showMenu();

    // Підказка
    cout << "|                Підказка:                |" << endl;
    cout << "+-----+" << endl;
    cout << "|        Введіть слово, для пошуку        |" << endl;
    cout << "|        /cancel – вихід без змін        |" << endl;
    cout << "+-----+" << endl;

    string keyword;
    cout << "Введіть слово для пошуку: ";
    getline(cin, keyword);

    if (keyword == "/cancel") {
        system("cls");
        showMenu();
        cout << "|                Пошук скасовано!                |" << endl;
        cout << "+-----+" << endl;
        return;
    }

    if (keyword.empty()) {
        system("cls");
        showMenu();
        cout << "|        Слово для пошуку не може бути        |" << endl;
        cout << "|                порожнім!                |" << endl;
        cout << "+-----+" << endl;
        return;
    }

    system("cls");
    showMenu();
    storage.searchMessages(keyword);
}

```

```

void deleteMessageFlow(MessageStorage& storage) {
    system("cls");
    showMenu();

    // 🐞 Перевірка: якщо чат порожній
    if (storage.getMessages().empty()) {
        cout << "|                Чат порожній!                |" << endl;
        cout << "|        Додайте повідомлення спочатку!        |" << endl;
        cout << "+-----+" << endl;
        return;
    }

    // Підказка
    cout << "|                Підказка:                |" << endl;
    cout << "+-----+" << endl;
    cout << "|        Введіть ID повідомлення, яке        |" << endl;
    cout << "|                бажаєте видалити                |" << endl;
    cout << "|        Введіть /cancel – вихід без змін        |" << endl;
    cout << "+-----+" << endl;

    string inputId;
    cout << "Введіть ID повідомлення для видалення: ";
    getline(cin, inputId);

    if (inputId == "/cancel") {

```

```

        system("cls");
        showMenu();
        cout << " |          Видалення скасовано          |" << endl;
        cout << "+-----+" << endl;
        return;
    }

    int id;
    try {
        id = stoi(inputId);
        if (id <= 0 || id > Message::getGlobalCounter()) {
            throw out_of_range("ID поза межами");
        }
    }
    catch (...) {
        int minId = 1;
        int maxId = Message::getGlobalCounter();
        system("cls");
        showMenu();
        cout << " |          Некоректний ID!          |" << endl;
        cout << " |          Введіть ID від " << minId << " до " << maxId << "          |" <<
endl;
        cout << "+-----+" << endl;
        return;
    }

    // Перевірка: чи існує повідомлення з таким ID
    bool exists = false;
    for (const auto& msg : storage.getMessages()) {
        if (msg->getId() == id) {
            exists = true;
            break;
        }
    }

    if (!exists) {
        int minId = 1;
        int maxId = Message::getGlobalCounter();
        system("cls");
        showMenu();
        cout << " |          Повідомлення з таким ID нема!          |" << endl;
        cout << " |          Введіть ID від " << minId << " до " << maxId << "          |" <<
endl;
        cout << "+-----+" << endl;
        return;
    }

    // Підтвердження
    string confirm;
    system("cls");
    showMenu();
    cout << " |          Ви впевнені, що хочете          |" << endl;
    cout << " |          видалити повідомлення з ID: " << id << "?          |" << endl;
    cout << " |          (Y – так ; N – ні ; /cancel)          |" << endl;
    cout << "+-----+" << endl;
    cout << "Ваш вибір: ";
    getline(cin, confirm);

    if (confirm == "/cancel" || confirm == "n" || confirm == "N") {
        system("cls");
        showMenu();
        cout << " |          Видалення скасовано          |" << endl;
        cout << "+-----+" << endl;
        return;
    }

```

```

    }

    if (confirm == "y" || confirm == "Y") {
        storage.deleteMessageById(id);
    }
    else {
        system("cls");
        showMenu();
        cout << "|          Некоректне підтвердження          |" << endl;
        cout << "+-----+" << endl;
    }
}

bool exitFlow(MessageStorage& storage) {

    string saveInput;
    cout << "Бажаєте зберегти перед виходом? (Y/N): ";
    getline(cin, saveInput);
    system("cls");
    showMenu();

    if (saveInput == "Y" || saveInput == "y") {
        storage.saveToFile();
    }
    else if (saveInput != "N" && saveInput != "n") {
        cout << "Некоректний вибір. Введіть Y або N " << endl;
        return false;
    }

    cout << "Вихід..." << endl;
    return true;
}

void refreshMenu() {
    system("cls");
    showMenu();
}

int main() {
    SetConsoleOutputCP(1251);
    SetConsoleCP(1251);
    setConsoleColor(8);

    MessageStorage storage;

    showMenu();
    while (true) {
        string input;
        int choice = -1;

        cout << "Виберіть дію: ";
        getline(cin, input);

        try {
            choice = stoi(input);
        }
        catch (...) {
            system("cls");
            showMenu();
            cout << "|          Некоректний вибір          |" << endl;
            cout << "|          Введіть число від 0 до 9          |" << endl;
            cout << "+-----+" << endl;
            continue;
        }
    }
}

```

```

    }

    if (choice < 0 || choice > 9) {
        system("cls");
        showMenu();
        cout << "| Введіть число в межах від 0 до 9 |" << endl;
        cout << "+-----+" << endl;
        continue;
    }

    switch (choice) {
    case 1:
        addMessageFlow(storage);
        break;
    case 2:
        refreshMenu();
        storage.displayMessages();
        break;
    case 3:
        refreshMenu();
        storage.saveToFile();
        break;
    case 4:
        refreshMenu();
        storage.loadFromFile();
        break;
    case 5: {editMessageFlow(storage); break;}
    case 6:
        refreshMenu();
        storage.clearMessages();
        break;
    case 7: {searchMessageFlow(storage); break;}
    case 8: {deleteMessageFlow(storage); break;}
    case 9:
        refreshMenu();
        storage.showStatistics();
        break;
    case 0: {if (exitFlow(storage)) return 0; break;}
    default:
        cout << "Некоректний вибір, спробуйте знову!" << endl;
    }
}
return 0;
}

```